

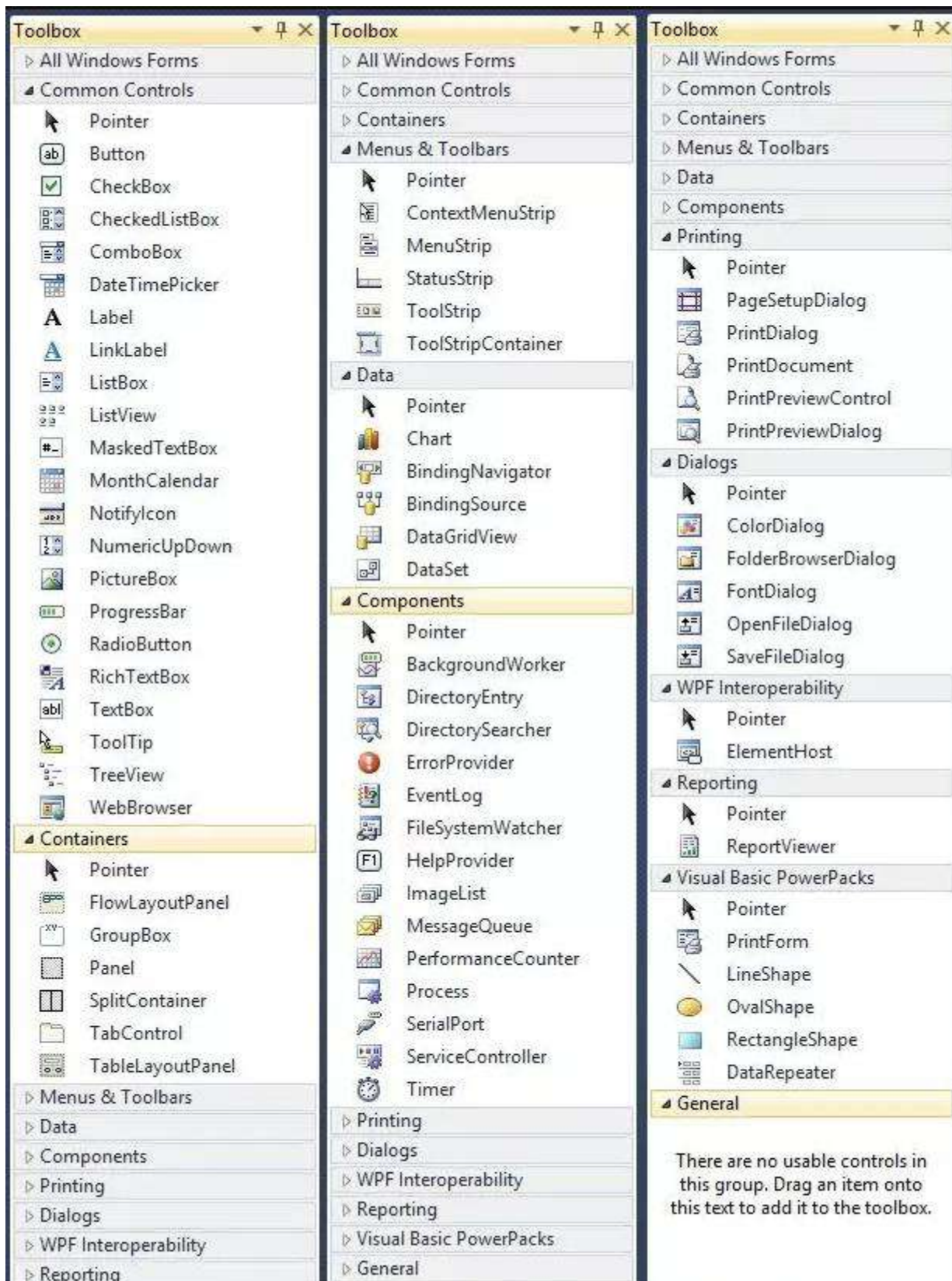
Unit – 3: Understanding Controls

Section – 1: .Net Common Controls:

In C# Windows Forms applications, ".NET common controls" refer to the standard set of user interface elements provided by the .NET Framework for building interactive desktop applications. These controls are readily available in the Visual Studio Toolbox when you are designing a Windows Forms application and are used to gather user input, display information, and provide functionality.

Common controls in WinForms typically fall into these main categories:

- **Input Controls:** Used to collect data or allow user interaction.
 - **TextBox:** For single or multi-line text input.
 - **Button:** To trigger actions when clicked.
 - **CheckBox:** To allow users to select or deselect an option.
 - **RadioButton:** To present a set of mutually exclusive choices.
 - **ComboBox:** A combination of a text box and a drop-down list.
 - **DateTimePicker:** For selecting dates and times.
- **Display Controls:** Used to present information to the user.
 - **Label:** To display static text.
 - **PictureBox:** To display images.
 - **ProgressBar:** To indicate the progress of an operation.
 - **ListView:** To display lists of items with various views (e.g., details, icons).
 - **DataGridView:** To display and edit tabular data.
- **Container/Layout Controls:** Used to organize and group other controls.
 - **Panel:** A general-purpose container for grouping controls.
 - **GroupBox:** A container with a border and title for grouping related controls.
 - **TabControl:** To organize content into multiple tabs.
 - **SplitContainer:** To divide a form into resizable panels.
- **Dialog Controls:** Used to open standard Windows dialogs for common tasks.
 - **OpenFileDialog:** To allow users to select a file to open.
 - **SaveFileDialog:** To allow users to select a location and name for saving a file.
 - **ColorDialog:** To allow users to choose a color.
 - **FontDialog:** To allow users to select a font.



To use these controls in a Windows Forms application:

- **Drag and Drop:**

From the Visual Studio Toolbox, drag the desired control onto your form in the design view.

- **Properties:**

Adjust the control's appearance and behavior using the Properties window (e.g., Text, Name, Location, Size).

- **Event Handling:**

Double-click on the control (or use the Events tab in the Properties window) to create event handlers (e.g., Click event for a Button) and write C# code to define the control's functionality.

Example (Button Click Event):

```
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("Button Clicked!");
}
```

Control Hierarchy:

The control hierarchy refers to the parent-child relationships between controls on a form. This hierarchy dictates how controls are visually organized and how events and properties are inherited and managed.

Core Principles of Control Hierarchy:

- **Inheritance from Control Class:**

All controls in System.Windows.Forms ultimately derive from the Control class. This base class provides fundamental properties (like Name, Text, Location, Size), events (like Click, Paint), and methods common to all UI elements.

- **Parent-Child Relationships:**

- Controls in a Windows Forms application are organized in a parent-child hierarchy.
- A Form is typically the top-level parent container for most controls.

- Controls can be nested within other container controls, such as Panel, GroupBox, TabControl, or SplitContainer. The control placed inside another control becomes a child of that container.
- The Controls collection of a parent control holds references to its direct child controls.
- A Panel or GroupBox can also act as a container for other controls, creating nested hierarchies.
- **Visual Layout and Positioning:**
 - A child control's position (Location) is relative to its parent container's client area.
 - Properties like Dock and Anchor on child controls help manage their automatic placement and resizing within their parent.
 - The Z-order (drawing order) of controls within a parent's Controls collection determines which controls are drawn on top of others.
- **Accessing Controls:**

You can navigate this hierarchy programmatically using the Controls collection of a container control. For example, `this.Controls` accesses controls directly on the form, while `panel1.Controls` accesses controls within `panel1`.
- **Event Bubbling and Handling:**
 - While events are typically handled directly by the control that raises them, some events can "bubble up" the control hierarchy, allowing parent controls to respond to events originating from their children.

Viewing the Control Hierarchy:

In Visual Studio, the Document Outline window (accessible via View > Other Windows > Document Outline or Ctrl + Alt + T) provides a tree-view representation of the control hierarchy within a selected form, making it easy to visualize and navigate the parent-child relationships.

.Net Common Controls:

1. Label:

The Label control is used to display static text (non-editable text) or images on a form. It is a foundational component for providing descriptions, instructions, or dynamic output. It is a non-

interactive control, meaning it does not typically respond to user input like clicks or key presses, though it does have a Click event.

Key characteristics and usage of the Label control:

- **Displaying Text:** The primary purpose of a Label is to display textual information to the user. This can include descriptions for other controls, output messages, instructions, or any other static text.

```
Label myLabel = new Label();  
myLabel.Text = "Enter your name:";  
myLabel.Location = new Point(10, 10);  
this.Controls.Add(myLabel);
```

- **Displaying Images:** While primarily for text, Label controls can also display images using the Image property, or a combination of ImageIndex and ImageList properties.

```
myLabel.Image = Image.FromFile("C:\\path\\to\\your\\image.jpg");
```

- **Properties for Customization:**

- Text: Sets or gets the text displayed by the label.
- Font: Specifies the font style, size, and type of the displayed text.
- ForeColor: Sets the color of the text.
- BackColor: Sets the background color of the label.
- AutoSize: If set to true, the label automatically adjusts its size to fit its content.
- TextAlign: Controls the alignment of the text within the label.
- Visible: Determines whether the label is visible on the form.
- Location, Height, Width, Size: Define the position and dimensions of the label on the form.

- **Accessibility (Access Keys):**

Labels can be used to provide access keys (mnemonics) for other controls, such as a TextBox. By setting the Target property of a Label to another control, pressing the access key (e.g., Alt + N for a label with &Name:) will move focus to the targeted control.

Adding a Label to a Form:

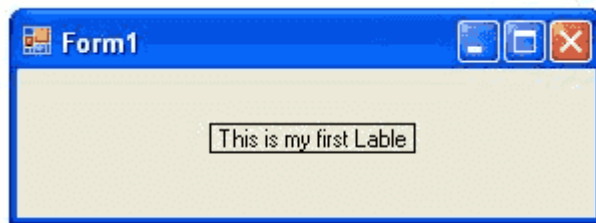
You can create and manipulate a label either visually at *design-time* in Visual Studio or programmatically at *run-time* using C# code.

- **Method 1: Drag and drop (Design-time) Using the Visual Studio Designer:**

Drag and drop the Label control from the Visual Studio Toolbox onto your Windows Form Designer. You can then adjust its properties in the Properties window.

Steps:

1. Open your Windows Forms project in Visual Studio.
2. Navigate to the **Toolbox**. If it's not visible, go to **View > Toolbox** or press **Ctrl+Alt+X**.
3. Find and click the **Label** control in the Toolbox.
4. Drag and drop the control onto your form in the designer window. A new **Label** control will appear on your form.



5. Select the label and open the **Properties** window (**F4**) to configure its settings. Here, you can change the **Text**, **Font**, **ForeColor** (text color), and other attributes.

- **Method 2: Create dynamically (Run-time) or Programmatically using C# code:**

Create an instance of the Label class, set its properties, and add it to the Controls collection of the parent form or container.

Create a label at run-time

You can also create and add a label to a form dynamically using C# code. This is useful for building dynamic interfaces based on user actions or data.

```
// In your form's code-behind file (e.g., Form1.cs), within a method like Form_Load.  
private void Form1_Load(object sender, EventArgs e)  
{
```

```
// Create a new Label object
Label myLabel = new Label();

// Set properties for the label
myLabel.Name = "dynamicLabel";
myLabel.Text = "This label was created with C# code.";
myLabel.Location = new Point(50, 50); // Sets the position on the form
myLabel.AutoSize = true; // Automatically resizes to fit its content

// Set other properties for styling
myLabel.Font = new Font("Arial", 10, FontStyle.Italic);
myLabel.ForeColor = Color.Blue;

// Add the label to the form's Controls collection
this.Controls.Add(myLabel);
}
```

Common label properties

You can access and modify these properties for a label at any point in your code. The following example assumes you have a label named `label1` on your form.

Property	Description	Example (in code)
Name	The unique identifier for the control. Used to reference it in code.	<code>label1.Name = "statusLabel";</code>
Text	The text that is displayed by the label.	<code>label1.Text = "Hello, World!";</code>
Font	Sets the font style, size, and type.	<code>label1.Font = new Font("Arial", 12, FontStyle.Bold);</code>
ForeColor	Sets the color of the text.	<code>label1.ForeColor = Color.Red;</code>
BackColor	Sets the background color of the label.	<code>label1.BackColor = Color.LightYellow;</code>

Visible	Determines whether the label is visible to the user.	<code>label1.Visible = false;</code>
Enabled	Determines if the label is enabled. Note: Labels are not interactive by default, so this property has limited use compared to other controls.	<code>label1.Enabled = true;</code>
Location	Sets the position of the label on the form.	<code>label1.Location = new Point(10, 10);</code>
Size	Sets the height and width of the control.	<code>label1.Size = new Size(150, 25);</code>
AutoSize	If <code>true</code> , the control automatically resizes to fit its contents.	<code>label1.AutoSize = true;</code>
TextAlign	Sets the alignment of the text within the label.	<code>label1.TextAlign = ContentAlignment.MiddleCenter;</code>

Common label events

While labels are not typically interactive, you can use certain events to react to user actions. You can subscribe to these events by double-clicking the label in the designer or by selecting the event from the **Properties** window.

Event	Description	Example (event handler)
Click	Occurs when the user clicks the label with the mouse.	<pre>private void label1_Click(object sender, EventArgs e) { // Code to execute on click }</pre>
DoubleClick	Occurs when the user double-clicks the label.	<pre>private void label1_DoubleClick(object sender, EventArgs e) { // Code to execute on double-click }</pre>

MouseEnter	Occurs when the mouse pointer enters the label's area.	<pre>private void label1_MouseEnter(object sender, EventArgs e) { // Code to execute on mouse hover }</pre>
MouseLeave	Occurs when the mouse pointer leaves the label's area.	<pre>private void label1_MouseLeave(object sender, EventArgs e) { // Code to execute when mouse leaves }</pre>
TextChanged	Occurs when the Text property is changed programmatically.	<pre>private void label1_TextChanged(object sender, EventArgs e) { // Code to execute on text change }</pre>

2. LinkLabel:

The **LinkLabel** control allows you to display text that functions as a hyperlink. It is often used to open a URL in a browser, trigger an email, or launch a file or another form within the application.

Key Features and Usage:

- **Adding a LinkLabel:**
 - **Method 1: Drag and drop (Design-time) Using the Visual Studio Designer:** Drag and drop the LinkLabel control from the Toolbox onto your Windows Form in the Visual Studio Designer.
 - **Method 2: Create dynamically (Run-time) or Programmatically using C# code:**

```
LinkLabel myLinkLabel = new LinkLabel();
this.Controls.Add(myLinkLabel);
```
- **Setting Properties:**
 - **Text:** Defines the visible text displayed by the LinkLabel.
 - **Name:** Used to refer to the control in your code.
 - **LinkArea:** Specifies the portion of the Text that acts as a clickable link. It takes a LinkArea object with a starting character position and the number of characters.

- **Links:** A collection of LinkLabel.Link objects, allowing you to define multiple hyperlinks within a single LinkLabel control. Each Link object can have its own LinkData (e.g., a URL or file path).
- **LinkColor, VisitedLinkColor, ActiveLinkColor:** Control the colors of the link in different states.
- **Handling Clicks:**
 - The LinkClicked event is the primary way to respond to a user clicking on the link.
 - Inside the LinkClicked event handler, you can access information about the clicked link through the LinkLabelLinkClickedEventArgs object, including its LinkData.
- **Implement the link action**

Inside the `LinkClicked` method, add the code for the desired action. The most common action is opening a web page using `System.Diagnostics.Process.Start()`.

Example of Opening a Web Page:

```
private void linkLabel1_LinkClicked(object sender, LinkLabelLinkClickedEventArgs e)
{
    // Assuming LinkData contains a URL
    if (e.Link.LinkData != null && e.Link.LinkData is string url)
    {
        System.Diagnostics.Process.Start(url); // Opens the URL in the default browser
        e.Link.Visited = true; // Mark the link as visited
    }
}
```

Example of Linking to another Form:

```
private void linkLabel2_LinkClicked(object sender, LinkLabelLinkClickedEventArgs e)
{
    // Assuming you have a form named "HelpForm"
    HelpForm helpForm = new HelpForm();
    helpForm.Show();
}
```

Advanced features and properties

Multiple links

You can have multiple hyperlinks within a single `LinkLabel` control. To do this, you use the `Links` property, which gives you access to the `LinkCollection`.

Example:

```
private void Form1_Load(object sender, EventArgs e)
{
    // Set the overall text for the LinkLabel
    linkLabel1.Text = "Click here to visit Microsoft or MSN.";

    // Add the first link for "Microsoft"
    // The link starts at index 23 and has a length of 9 characters.
    linkLabel1.Links.Add(23, 9, "https://www.microsoft.com");

    // Add the second link for "MSN"
    // The link starts at index 36 and has a length of 3 characters.
    linkLabel1.Links.Add(36, 3, "https://www.msn.com");
}

private void linkLabel1_LinkClicked(object sender, LinkLabelLinkClickedEventArgs e)
{
    // Cast the LinkData to a string (the URL)
    string target = e.Link.LinkData as string;

    // Open the appropriate URL
    if (target != null)
    {
        System.Diagnostics.Process.Start(target);
        e.Link.Visited = true; // Mark the specific link as visited
    }
}
```

Appearance

You can change the colors of the link depending on its state by using these properties:

- `LinkColor` : The default color of the link.
- `VisitedLinkColor` : The color after the user has clicked the link.
- `ActiveLinkColor` : The color when the user is actively clicking the link.
- `DisabledLinkColor` : The color when the link is disabled.

You can also change how the link is underlined with the `LinkBehavior` property, which can be set to `AlwaysUnderline` , `HoverUnderline` , or `NeverUnderline` .

3. TextBox:

The `TextBox` is a fundamental control for C# Windows Forms applications, used to display and accept user input. You can add and configure a `TextBox` either visually in the Forms Designer or dynamically through code.

Creating a TextBox:

There are two primary ways to add a `TextBox` to a Windows Forms application.

- **Method 1: Drag and drop (Design-time) Using the Visual Studio Designer::**

Drag and drop a `TextBox` control from the Visual Studio Toolbox onto your Windows Form. You can then position and resize it as needed.

Using Visual Studio:

1. In Visual Studio, open your Windows Form in the designer view.
 2. Navigate to the **Toolbox** panel (View > Toolbox).
 3. Expand the **Common Controls** group.
 4. Drag and drop a `TextBox` control onto your form.
 5. With the control selected, use the **Properties** window to configure it.
- **Method 2: Create dynamically (Run-time) or Programmatically using C# code:**
Instantiate a `TextBox` object in your C# code, set its properties, and add it to the form's Controls collection.

Dynamically with code:

1. Create a new `TextBox` object in your form's code.
2. Set its properties, such as `Name` , `Location` , and `Size` .
3. Add the new `TextBox` to the form's `Controls` collection.

Example code

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
        CreateTextBoxAtRunTime();
    }

    private void CreateTextBoxAtRunTime()
    {
        // Create a new TextBox instance
        TextBox myTextBox = new TextBox();

        // Set its properties
        myTextBox.Name = "dynamicTextBox";
        myTextBox.Text = "Hello World!";
        myTextBox.Location = new Point(50, 50); //Position it at (50, 50)
        myTextBox.Size = new Size(200, 25);

        // Add the TextBox to the form's controls
        this.Controls.Add(myTextBox);
    }
}
```

Key properties

You can customize the behavior and appearance of a `TextBox` using its properties.

Property	Description
<code>Text</code>	Gets or sets the current text in the text box. This is the most used property for retrieving or setting content.

Name	Gets or sets the name used to identify the control in code.
Multiline	A boolean value that, when set to <code>true</code> , allows the <code>TextBox</code> to accept and display multiple lines of text.
ReadOnly	A boolean value that makes the <code>TextBox</code> non-editable by the user. The text can still be set or retrieved programmatically.
PasswordChar	Gets or sets the character used to mask input, such as an asterisk (<code>*</code>), for password entry.
MaxLength	Sets the maximum number of characters that the user can type into the text box.
ScrollBars	Specifies if and which scroll bars (horizontal, vertical, or both) should appear when the <code>Multiline</code> property is enabled and the text exceeds the visible area.
CharacterCasing	Specifies whether the characters typed by the user are converted to lowercase or uppercase.
TextAlign	Sets the alignment of the text, such as <code>Left</code> , <code>Center</code> , or <code>Right</code> .
Visible	A boolean value to show or hide the control.
Enabled	A boolean property that, when false, disables the <code>TextBox</code> , making it uneditable and visually grayed out.
BackColor / ForeColor	Control the background and foreground colors of the <code>TextBox</code> .
TextChanged Event	Fires whenever the <code>Text</code> property of the <code>TextBox</code> changes, either by user input or programmatically.
Validated / Validating Events	Useful for implementing input validation logic.

Important events

Events are used to respond to user interactions with the `TextBox`.

Event	Description
<code>TextChanged</code>	Occurs every time the <code>Text</code> property changes, which happens with every keystroke. This is useful for real-time validation or updates.
<code>Validating</code>	Occurs when the control is about to lose focus and the application needs to validate the content. You can cancel the event to keep the focus on the <code>TextBox</code> until the input is correct.
<code>KeyDown</code>	Occurs when a key is pressed while the control has focus. This can be used to handle specific key presses, such as the <code>Enter</code> key.
<code>Enter</code>	Occurs when the <code>TextBox</code> receives focus.
<code>Leave</code>	Occurs when the <code>TextBox</code> loses focus.

Example: Handling events

This example demonstrates a common scenario: validating user input after the user leaves the `TextBox`.

Forms Designer steps:

1. Drag a `TextBox` and an `ErrorProvider` control onto your form.
2. Set the `ErrorProvider`'s `Name` to `errorProvider1` and the `TextBox`'s `Name` to `textBoxAge`.
3. In the Properties window for `textBoxAge`, set `AutoValidate` to `EnableAllowFocusChange` or `EnablePreventFocusChange`.
4. Double-click the `textBoxAge`'s `Validating` event in the Properties window to automatically generate the event handler.

Code (Form1.cs):

```
private void textBoxAge_Validating(object sender, System.ComponentModel.CancelEventArgs e)
{
    // Try to parse the text as an integer
    if (!int.TryParse(textBoxAge.Text, out int age) || age < 0)
    {
        // Invalid input: Set an error and cancel the event
        errorProvider1.SetError(textBoxAge, "Please enter a valid, non-negative number.");
        e.Cancel = true; // Prevents the user from leaving the textbox
    }
    else
    {
        // Valid input: Clear the error
        errorProvider1.SetError(textBoxAge, "");
    }
}
```

Common Operations:

- **Getting/Setting Text:** Access the Text property.
- **Clearing Text:** Use the Clear() method or set Text to an empty string.
- **Appending Text:** Use the AppendText() method for multiline TextBoxes.
- **Setting Focus:** Use the Focus() method to place the cursor in the TextBox.

4. Rich Text Box:

The `RichTextBox` control in C# Windows Forms applications is an advanced text editor that enables the display and manipulation of text with advanced formatting capabilities beyond a standard `TextBox`. It allows for the display, entry, and manipulation of text with various formatting options, including fonts, colors, and links. It is more powerful than the standard `TextBox` control, which only handles plain text. It can also load and save Rich Text Format (RTF) files and handle embedded images.

Key Features and Usage:

- **How to use the RichTextBox control:**

There are two primary ways to add a `RichTextBox` to your application in Visual Studio:

- **Method 1: Drag and drop (Design-time) Using the Visual Studio Designer:** Drag and drop the `RichTextBox` control from the Visual Studio Toolbox onto your Windows Form.

This is the simplest way to add the control.

1. In Visual Studio, create a new **Windows Forms App (.NET Framework)** project.
 2. Open the **Form Designer**.
 3. From the **Toolbox**, drag and drop a `RichTextBox` control onto your form.
 4. Use the **Properties** window to configure the control's appearance and behavior, such as its size, location, and border style.
- **Method 2: Create dynamically (Run-time) or Programmatically using C# code:** Instantiate the `RichTextBox` control in your code and add it to the form's Controls collection:

```
// Create a RichTextBox object
RichTextBox myRichTextBox = new RichTextBox();

// Set properties
myRichTextBox.Location = new Point(10, 10);
myRichTextBox.Size = new Size(300, 200);
myRichTextBox.Text = "Hello, World!";
myRichTextBox.Multiline = true;
myRichTextBox.ScrollBars = RichTextBoxScrollBars.Both;

// Add the RichTextBox to the form's Controls collection
this.Controls.Add(myRichTextBox);
```

- **Adding Text:**

- **Design-Time:** Set the Text property in the Properties window.
- **Run-Time:** Assign a string to the Text property or use the `AppendText()` method to add text programmatically:
Code

```
myRichTextBox.Text = "This is some initial text.";
myRichTextBox.AppendText("\nThis is a new line of text.");
```

Key features and code examples

Basic text manipulation

You can use the `Text` and `AppendText()` methods to add or modify text. For unformatted text, use the `Text` property. For formatted text or appends, it's better to use `SelectedText` after setting formatting options.

```
// Append new text to the end
richTextBox1.AppendText("This is a new line.\r\n");

// Clear all text
richTextBox1.Clear();
```

Formatting selected text

To apply formatting to specific portions of text, you must select the text first by setting the `SelectionStart` and `SelectionLength` properties. You can then modify the text's formatting attributes.

```
private void formatButton_Click(object sender, EventArgs e)
{
    // Find the word "important"
    int startIndex = richTextBox1.Find("important", RichTextBoxFinds.None);

    if (startIndex != -1)
    {
        // Select the text and apply formatting
        richTextBox1.SelectionStart = startIndex;
        richTextBox1.SelectionLength = "important".Length;
        richTextBox1.SelectionColor = Color.Red;
        richTextBox1.SelectionFont = new Font(richTextBox1.Font, FontStyle.Bold);
    }
}
```

Loading and saving files

The `RichTextBox` can handle Rich Text Format (.rtf) and plain text files with built-in methods.

```
// Load an RTF file into the control
richTextBox1.LoadFile(@"C:\path\to\document.rtf", RichTextBoxStreamType.RichText);

// Save the content to an RTF file
```

```
richTextBox1.SaveFile(@"C:\path\to\new_document.rtf", RichTextBoxStreamType.RichText);
```

Working with RTF content

The `Rtf` property lets you get or set the control's entire content, including formatting, as a single string.

```
// Store formatted content in a string
string rtfContent = richTextBox1.Rtf;

// Load RTF content from a string
string newRtf = @"{\rtf1\ansi {\fonttbl {\f0\fswiss\fcharset0 Arial;}} \par This is {\b bold} text.}";
richTextBox1.Rtf = newRtf;
```

Enabling URLs

To automatically turn text URLs into clickable links, set the `DetectUrls` property to `true`.

```
// Enable automatic URL detection
richTextBox1.DetectUrls = true;
```

To handle the clicks, you must subscribe to the `LinkClicked` event.

```
// Event handler for a link click
private void richTextBox1_LinkClicked(object sender, LinkClickedEventArgs e)
{
    // Handle the link click, for example, by opening it in a browser
    System.Diagnostics.Process.Start(e.LinkText);
}
```

RichTextBox vs. TextBox

Feature	RichTextBox	TextBox
Formatting	Yes. Supports fonts, colors, styles (bold, italics, underline), and indentation.	No. Only supports a single font for all text.
Data type	Handles Rich Text Format (RTF) data, which includes formatting information.	Handles plain text (strings) only.

Resource usage	More resource-intensive due to advanced features.	Less resource-intensive, making it ideal for simple text input.
Images	Can display inline images.	Cannot display images.
Character limit	No character limit by default, though performance may degrade with very large files.	Limited to 64 KB of text.
Use case	Long, formatted documents, logging with color-coding, or email clients.	Simple, unformatted data input, like user names or short comments.

Other Useful Features:

- **Undo/Redo:** Built-in functionality for editing operations.
- **Find:** Methods to search for specific text within the control.
- **Scroll Bars:** Displays both horizontal and vertical scrollbars by default, with customizable settings through the ScrollBars property.

Events: Handles various events like TextChanged for responding to content changes.

The RichTextBox is a versatile control for implementing features similar to word processing applications within your C# Windows Forms applications.

5. Picture Box:

The **PictureBox** control in C# Windows Forms is used to display images in various formats, such as bitmaps, JPEGs, and GIFs. It provides a simple way to incorporate visual elements like photographs, icons, or graphics into your desktop applications. You can load images into it at design time via the Properties window or dynamically at runtime using code.

Key Features and Usage:

Displaying Images:

- The primary function is to display images in various formats, including BMP, GIF, JPEG, PNG, and icons.
- You can set the image at design time (using the Properties window in Visual Studio) or at runtime programmatically.

Setting Images:

- **Method 1: Drag and drop (Design-time) Using the Visual Studio Designer:**

1. Drag and drop the `PictureBox` control from the **Toolbox** onto your Windows Form.
2. Select the `PictureBox` and go to the **Properties** window.
3. Click the ellipsis (...) next to the **Image** property.
4. In the "Select Resource" dialog, you can either import a local image file or select a resource from your project.

- **Method 2: Create dynamically (Run-time) or Programmatically using C# code:**

You can load an image from a local file or a URL by setting the `Image` or `ImageLocation` property.

// Load an image from a file

```
pictureBox1.Image = Image.FromFile("C:\\Path\\To\\Your\\Image.jpg");
```

// Load an image from a Bitmap object

```
Bitmap myBitmap = new Bitmap("C:\\Path\\To\\Your\\AnotherImage.png");  
pictureBox1.Image = myBitmap;
```

// Load image from a local file

```
pictureBox1.Image = Image.FromFile(@"C:\path\to\your\image.jpg");
```

// Load image from a URL (asynchronously)

```
pictureBox1.ImageLocation = "https://example.com/image.png";
```

Note: Using `LoadAsync()` is recommended for large images to prevent your application's UI from freezing while the image loads.

Image Scaling and Sizing:

- The **SizeMode** property controls how the image is displayed within the `PictureBox`:
- **Normal**: Displays the image at its original size, starting from the top-left corner.
- **StretchImage**: Stretches or shrinks the image to fit the `PictureBox`'s dimensions.
- **AutoSize**: Resizes the `PictureBox` to fit the image's original dimensions.
- **CenterImage**: Centers the image within the `PictureBox` without scaling.
- **Zoom**: Scales the image to fit within the `PictureBox` while maintaining its aspect ratio.

Key properties

Property	Description
Image	Gets or sets the image that is displayed in the control. You can assign an <code>Image</code> object (like a <code>Bitmap</code>) to this property.
ImageLocation	Gets or sets the path or URL of the image to display. The image will be loaded from this location.
SizeMode	Controls how the image is positioned and scaled inside the picture box. The most common values are: Normal : Places the image in the top-left corner. Clips the image if it is larger than the control. StretchImage : Stretches or shrinks the image to fit the control's dimensions, which can distort the image. AutoSize : Resizes the control to match the size of the image. CenterImage : Centers the image within the control, clipping it if it's too large. Zoom : Resizes the image to fit the control while maintaining the aspect ratio.
BorderStyle	Sets the border style for the control (e.g., <code>FixedSingle</code> , <code>Fixed3D</code> , or <code>None</code>).
ErrorImage	Gets or sets the image to display if an error occurs during the image-loading process.
InitialImage	Gets or sets the temporary image that is displayed while the main image is loading.
WaitOnLoad	Determines whether the image loads synchronously (blocking the UI) or asynchronously.

Important events

The `PictureBox` control exposes various events, such as `Click`, `DoubleClick`, `MouseEnter`, `MouseLeave`, which allow you to respond to user interactions with the displayed image.

- **Click** and **DoubleClick**: Triggered when the user clicks or double-clicks the control. You can add an event handler to perform an action, such as showing a message box or opening a file dialog.

- **MouseEnter** and **MouseLeave** : Occur when the mouse cursor enters or leaves the bounds of the control. These are useful for creating interactive hover effects.
- **LoadCompleted** : Occurs after an asynchronous image load finishes. This is useful when using the **LoadAsync** method.

Example of Basic Usage:

```
// In your Form's constructor or a button click event handler
private void Form1_Load(object sender, EventArgs e)
{
    // Assuming you have a PictureBox named 'pictureBox1' on your form
    // Load an image from a file
    pictureBox1.Image = Image.FromFile("C:\\Users\\Public\\Pictures\\Sample
Pictures\\Koala.jpg");

    // Set the SizeMode to Zoom to fit the image while maintaining aspect ratio
    pictureBox1.SizeMode = PictureBoxSizeMode.Zoom;
}
```

Example: Loading and resizing an image dynamically

This example shows how to use a button to open a file dialog, load the selected image into a **PictureBox** , and set its **SizeMode** .

Steps

1. Create a new Windows Forms project.
2. Add a **PictureBox** control and a **Button** control to your form.
3. Double-click the **Button** to generate its **Click** event handler in the code-behind file.
4. Add the following code to the **button1_Click** method:

```
private void button1_Click(object sender, EventArgs e)
{
    // Create an OpenFileDialog object
    OpenFileDialog openFileDialog1 = new OpenFileDialog();

    // Set the file filter
```

```

openFileDialog1.Filter = "Image Files|*.jpg;*.jpeg;*.png;*.gif;*.bmp";
openFileDialog1.Title = "Select an Image File";

// Show the dialog and get the result
if (openFileDialog1.ShowDialog() == DialogResult.OK)
{
    try
    {
        // Set the PictureBox's image to the selected file
        pictureBox1.Image = new Bitmap(openFileDialog1.FileName);

        // Set the SizeMode to Zoom to fit the image
        pictureBox1.SizeMode = PictureBoxSizeMode.Zoom;
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error loading image: " + ex.Message);
    }
}
}

```

6. Button:

A C# button is a standard Windows Forms control that allows users to initiate an action within an application. The Button control is a fundamental element for user interaction, allowing users to initiate actions by clicking.

Adding a Button:

You can add buttons in two ways: using the Visual Studio Designer or by writing code to create them at runtime.

Method 1: Drag and drop (Design-time) Using the Visual Studio Designer:

Adding a button using the designer

1. **Open the Toolbox:** In Visual Studio, go to the menu and select **View > Toolbox**. You can also use the shortcut `Ctrl+Alt+X`.

2. **Drag and drop:** Find the **Button** control in the "Common Controls" section of the Toolbox and drag it onto your form.
3. **Set properties:** With the button selected, use the **Properties** window (`F4`) to customize its appearance and behavior. Common properties include:
 - **(Name)** : The variable name used to reference the button in your code (e.g., `btnSubmit`).
 - **Text** : The text that appears on the button (e.g., "Submit").
 - **Location** : The position of the button on the form.
 - **Size** : The dimensions of the button.
 - **Enabled** : A boolean value that determines if the button is clickable.
 - **Visible** : A boolean value that controls the button's visibility.

Method 2: Create dynamically (Run-time) or Programmatically using C# code:

1. Create an instance of the Button class:

```
Button myButton = new Button();
```
2. Set properties like Text, Location, Size, etc.:

```
myButton.Text = "Click Me";  
myButton.Location = new Point(100, 50);  
myButton.Size = new Size(80, 30);
```
3. Add the button to the form's Controls collection:

```
this.Controls.Add(myButton);
```

Handling Button Clicks (Events):

The primary purpose of a button is to trigger an action when clicked. This is achieved by handling its Click event. To make your button perform a task, you must create an event handler for its Click event.

- **Method 1: Generate the event handler - Design-time (using Visual Studio Designer):**
 - Select the button on your form.

- Double-click the button or navigate to the Events section (lightning bolt icon) in the Properties window.
- Double-click the Click event to automatically generate an event handler method in your code-behind file.

Add code: Visual Studio will automatically generate a new method in your code-behind file. Add the code you want to execute when the button is clicked inside this method.

Example:

```
private void btnSubmit_Click(object sender, EventArgs e)
{
    // This code will run when the "Submit" button is clicked.
    MessageBox.Show("Button was clicked!");
}
```

- **Create dynamically (Run-time) or Programmatically using C# code::**

- Attach an event handler to the Click event:
myButton.Click += new EventHandler(MyButton_Click);
- Define the event handler method.

```
private void MyButton_Click(object sender, EventArgs e)
{
    // Code to execute when the button is clicked
    MessageBox.Show("Button clicked!");
}
```

You can also create a button and its event handler entirely in code, which is useful for creating dynamic user interfaces.

Example:

```
using System;
using System.Drawing;
using System.Windows.Forms;

public partial class Form1 : Form
{
    public Form1()
```

```

{
    InitializeComponent();
    CreateDynamicButton();
}

private void CreateDynamicButton()
{
    // 1. Create a new Button instance
    Button dynamicButton = new Button();

    // 2. Set the button's properties
    dynamicButton.Name = "dynamicButton";
    dynamicButton.Text = "Click Me";
    dynamicButton.Location = new Point(50, 50);
    dynamicButton.Size = new Size(150, 40);

    // 3. Attach the click event handler
    dynamicButton.Click += new EventHandler(DynamicButton_Click);

    // 4. Add the button to the form's controls
    this.Controls.Add(dynamicButton);
}

// This is the event handler for the dynamically created button
private void DynamicButton_Click(object sender, EventArgs e)
{
    MessageBox.Show("Dynamic button was clicked!");
}
}

```

Common Button Properties:

- Text: The text displayed on the button.
- Name: The programmatic name of the button (used to refer to it in code).
- Enabled: A boolean value indicating whether the button is interactive (true) or disabled (false).
- Visible: A boolean value indicating whether the button is visible (true) or hidden (false).

- **Font:** Controls the font style, size, and other text attributes.
- **Image:** Allows displaying an image on the button.
- **ImageAlign:** Specifies the alignment of the image relative to the text.
- **TextAlign:** Specifies the alignment of the text within the button.

7. GroupBox:

A GroupBox in a C# Windows Forms application is a container control used to visually and logically group related controls on a form. It provides a border and an optional title (text) to enclose its child controls, enhancing the organization and user experience of your application.

Key Features and Usage:

- **Logical Grouping:**

GroupBoxes are primarily used to group controls that share a common purpose or functionality. For example, you might place a group of RadioButton controls related to a single selection within a GroupBox, or input fields for an address.

- **Visual Organization:**

They provide a clear visual boundary around the grouped controls, making the form easier to understand and navigate for the user.

- **Container Properties:**

As a container, a GroupBox can inherit properties from its parent form and can also pass down certain properties (like Enabled or Font) to its contained controls. If you disable a GroupBox, all controls within it will also become disabled.

Adding Controls:

- **Method 1: Drag and drop (Design-time) Using the Visual Studio Designer:**

1. Open your form in the Visual Studio designer.
2. From the **Toolbox**, drag and drop a `GroupBox` onto your form.
3. Drag and drop other controls (such as `RadioButtons` or `CheckBoxes`) into the `GroupBox`.

4. Use the **Properties** window to customize the `GroupBox`. You can set the `Text` property for the title and adjust its `Location` and `Size`.

- **Method 2: Create dynamically (Run-time) or Programmatically using C# code:** You can add controls programmatically to a `GroupBox` using its `Controls.Add()` method.

```
// Example of adding a Button to a GroupBox at runtime
Button myButton = new Button();
myButton.Text = "Click Me";
// Position relative to the GroupBox

myButton.Location = new Point(10, 20);
myGroupBox.Controls.Add(myButton);
```

Setting the Title: The title displayed at the top of the `GroupBox` is set using its `Text` property.

```
myGroupBox.Text = "User Options";
```

Example:

This code creates a `GroupBox` with two radio buttons and adds them to a form.

```
csharp
using System.Drawing;
using System.Windows.Forms;

public class MyForm : Form
{
    public MyForm()
    {
        InitializeComponent();
    }

    private void InitializeComponent()
    {
        // Create the GroupBox
        GroupBox genderGroup = new GroupBox();
        genderGroup.Text = "Select Gender";
        genderGroup.Location = new Point(20, 20);
        genderGroup.Size = new Size(200, 100);

        // Create RadioButtons
```

```

RadioButton maleRadio = new RadioButton();
maleRadio.Text = "Male";
maleRadio.Location = new Point(10, 20);

RadioButton femaleRadio = new RadioButton();
femaleRadio.Text = "Female";
femaleRadio.Location = new Point(10, 45);

// Add the RadioButtons to the GroupBox's Controls collection
genderGroup.Controls.Add(maleRadio);
genderGroup.Controls.Add(femaleRadio);

// Add the GroupBox to the form's Controls collection
this.Controls.Add(genderGroup);

this.Text = "GroupBox Example (WinForms)";
this.Size = new Size(250, 170);
}
}

```

Example:

This code creates and configures a `GroupBox` and its contents in the code-behind file.

```

csharp
using System.Windows;
using System.Windows.Controls;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        CreateGroupBoxInCode();
    }

    private void CreateGroupBoxInCode()
    {
        // Create the GroupBox
    }
}

```

```

GroupBox groupBox = new GroupBox();
groupBox.Header = "Product Details";
groupBox.Margin = new Thickness(10);

// Create a StackPanel to hold multiple controls
StackPanel stackPanel = new StackPanel();
stackPanel.Margin = new Thickness(10);

// Create and add controls to the StackPanel
stackPanel.Children.Add(new TextBlock { Text = "Product Name:" });
stackPanel.Children.Add(new TextBox { Margin = new Thickness(0, 5, 0, 0) });

// Set the StackPanel as the single child of the GroupBox
groupBox.Content = stackPanel;

// Add the GroupBox to the main Grid of the window (assuming your XAML has a Grid named
`MyGrid`)
// MyGrid.Children.Add(groupBox);
// For this example, we add it to a new window directly
var window = new Window();
var mainGrid = new Grid();
mainGrid.Children.Add(groupBox);
window.Content = mainGrid;
window.Show();
}
}

```

Example Scenario:

Imagine a form where you need to collect user preferences. You could use a GroupBox titled "Notification Settings" to contain checkboxes for email, SMS, and push notifications, and another GroupBox titled "Theme Selection" to contain radio buttons for light and dark themes. This organization makes the form more intuitive and user-friendly.

8. Panel:

The `Panel` control is a container control used to group other controls. It is an essential tool for organizing your UI, allowing you to manage a collection of related controls as a single unit.

Key Features and Usage:

- **Container for Controls:**

Panels are designed to hold and organize multiple controls, such as buttons, text boxes, labels, and other container controls. This facilitates the management and manipulation of related UI elements.

- **Logical Grouping and Organization:**

Place related controls (like a set of radio buttons or form fields) inside a `Panel` to provide visual organization. This improves the user experience and makes the application more intuitive.

- **Layout Management:**

Panels can aid in creating responsive layouts. You can use properties like `Dock` and `Anchor` on controls within a `Panel` to control their positioning and resizing behavior relative to the `Panel`.

- **Show and hide groups:** Rather than showing or hiding individual controls, you can simply change the `Visible` property of the `Panel` to hide or display all of its child controls at once.

- **Visibility and Enabled State:**

If the `Enabled` property of a `Panel` is set to `false`, all controls contained within that `Panel` will also be disabled. Similarly, you can hide or show an entire group of controls by changing the `Visible` property of the `Panel`.

- **Scrolling Capability:**

Panels derive from the `ScrollableControl` class, meaning they can provide scroll bars if the content within them exceeds the `Panel`'s visible area. Unlike a `GroupBox`, a `Panel` can be configured to show scroll bars. This is useful for displaying a large number of controls in a limited space by setting the `AutoScroll` property to `true`.

- **Borders and Appearance:**

While Panels are borderless by default, you can add a border using the `BorderStyle` property (e.g., `FixedSingle`, `Fixed3D`) to visually distinguish the Panel's boundaries.

- **Dynamic Content:**

Panels are often used when you need to dynamically add or remove controls at runtime, or when switching between different "pages" or views within a single form by showing and hiding different Panels.

- **Dynamic layout:** The `Panel` can be used as a dynamic placeholder. For example, you can create a tabbed interface by using multiple `Panel` controls and bringing the appropriate one to the front.

How to use a `Panel` control

Method 1: Drag and drop (Design-time) Using the Visual Studio Designer:

1. Open your form in the Visual Studio designer.
2. Find the `Panel` control in the **Toolbox**.
3. Drag and drop the `Panel` onto your form.
4. Drag and drop other controls (such as `TextBox` and `Label`) onto the `Panel`. These controls will now be "children" of the `Panel`.
5. Set properties in the **Properties** window, such as `BorderStyle` or `AutoScroll`, to configure the panel's appearance and behavior.

Method 2: Create dynamically (Run-time) or Programmatically using C# code:

You can create and add a `Panel` and its controls dynamically.

```
// Create a new Panel object
Panel myPanel = new Panel();
myPanel.Location = new Point(10, 10);
myPanel.Size = new Size(200, 150);
myPanel.BorderStyle = BorderStyle.FixedSingle;
myPanel.AutoScroll = true;
```

```
// Create a new Label and add it to the Panel
Label myLabel = new Label();
myLabel.Location = new Point(10, 10);
myLabel.Text = "This is a label inside a panel.";
myPanel.Controls.Add(myLabel);

// Add the Panel to the form's Controls collection
this.Controls.Add(myPanel);
```

Important properties

- **Visible** : Gets or sets a value indicating whether the panel and its child controls are displayed.
- **AutoScroll** : Gets or sets a value indicating whether scroll bars appear when the panel's contents are larger than its visible area.
- **BorderStyle** : Specifies the border style for the panel. Common values are **None** (default), **FixedSingle**, and **Fixed3D**.
- **Dock** : Specifies which control borders are docked to its parent control.
- **Controls** : The collection of controls contained within the panel. You can add or remove child controls at runtime using this property.

Panel vs. GroupBox

The **Panel** control is similar to the **GroupBox**, but there are a few key differences:

- **Caption**: A **GroupBox** can display a text caption, while a **Panel** cannot.
- **Scroll bars**: A **Panel** has an **AutoScroll** property that enables scroll bars. A **GroupBox** does not have this feature.
- **Default border**: A **GroupBox** always displays a border by default, while a **Panel**'s **BorderStyle** is **None** by default.

Example of Adding Controls to a Panel Programmatically:

```
private void AddControlsToPanel()
{
    // Create a new Panel
    Panel dynamicPanel = new Panel();
    dynamicPanel.Location = new System.Drawing.Point(50, 50);
    dynamicPanel.Name = "dynamicPanel";
    dynamicPanel.Size = new System.Drawing.Size(300, 200);
    dynamicPanel.BackColor = System.Drawing.Color.LightGray;
    this.Controls.Add(dynamicPanel); // Add the panel to the form

    // Create a TextBox and add it to the Panel
    TextBox textBox1 = new TextBox();
    textBox1.Location = new System.Drawing.Point(20, 20);
    textBox1.Text = "Hello from Panel!";
    dynamicPanel.Controls.Add(textBox1);

    // Create a Button and add it to the Panel
    Button button1 = new Button();
    button1.Location = new System.Drawing.Point(20, 60);
    button1.Text = "Click Me";
    dynamicPanel.Controls.Add(button1);
}
```

9. CheckBox:

The CheckBox control allows users to select or deselect a boolean-type option (e.g., Yes/No, True/False) choice and is located in the `System.Windows.Forms` namespace. Unlike radio buttons, multiple checkboxes can be selected simultaneously.

Creating a CheckBox:

You can create and manage checkboxes at design time using the Visual Studio designer, or dynamically at run time with code.

- **Method 1: Drag and drop (Design-time) Using the Visual Studio Designer:**
 1. In Visual Studio, open a Windows Forms project and navigate to the form designer view.
 2. Open the **Toolbox** (**View > Toolbox** or press `Ctrl + Alt + X`).
 3. Drag a `CheckBox` control from the "Common Controls" section and drop it onto your form.

4. With the control selected, use the **Properties** window to configure its appearance and behavior. Common properties to set include `Text`, `Name`, `Location`, `Size`, and `Checked`.

- **Method 2: Create dynamically (Run-time) or Programmatically using C# code:**

1. Create an instance of the `CheckBox` class.
2. Set its properties (e.g., `Text`, `Location`, `Checked`).
3. Add the checkbox to the form's controls collection.

// Create a CheckBox object

```
CheckBox dynamicCheckBox = new CheckBox();
```

// Set properties

```
dynamicCheckBox.Text = "Agree to terms";
```

```
dynamicCheckBox.Location = new Point(50, 50);
```

```
dynamicCheckBox.Checked = false; // Initial state
```

// Add to the form

```
this.Controls.Add(dynamicCheckBox);
```

Key Properties:

- `Checked`: A `bool` value that indicates whether the checkbox is checked (`true`) or not (`false`). This is the most common property to check.
- `CheckState`: A more robust alternative to `Checked`, this property of the `CheckState` enumeration can be `Checked`, `Unchecked`, or `Indeterminate`. The `Indeterminate` state is enabled by setting the `ThreeState` property to `true`.
- `ThreeState`: A `bool` that determines if the checkbox can have three states instead of two.
- `Text`: The string that appears next to the checkbox.
- `Name`: A unique identifier for the control, used to reference it in your code.
- `Appearance`: An enumeration that can be set to `Normal` (standard checkbox) or `Button` (a toggle button).

- **Enabled** : A boolean determining if the user can interact with the control.
- **Visible** : A boolean determining if the checkbox is displayed on the form.

Important events

- **CheckedChanged** : Fired when the value of the **Checked** property / Checked state of the checkbox changes. This is the most common event for responding to a user's selection / handling checkbox interactions.

Handling the **CheckedChanged** event

This event is crucial for making your application react to the user's input. The standard procedure is to double-click the checkbox in the design view to automatically generate a method for its **CheckedChanged** event.

Example:

Let's say you have a **CheckBox** named **checkBox1** and a **Button** named **button1** on your form. The following code will enable or disable the button based on the checkbox's state.

```
private void myCheckBox_CheckedChanged(object sender, EventArgs e)
{
    if (myCheckBox.Checked)
    {
        MessageBox.Show("Checkbox is checked!");
    }
    else
    {
        MessageBox.Show("Checkbox is unchecked!");
    }
}
```

- **CheckStateChanged** : Fired when the value of the **CheckState** property changes.

Working with multiple checkboxes

If you have a group of checkboxes, you can iterate through them to determine which ones are selected. One way to do this is to add them to a `GroupBox` control and loop through the controls collection.

Example:

```
// Assuming multiple checkboxes are in a GroupBox named 'groupBox1'
private void button1_Click(object sender, EventArgs e)
{
    string selectedItems = "Selected items: ";
    foreach (Control control in groupBox1.Controls)
    {
        if (control is CheckBox checkbox && checkbox.Checked)
        {
            selectedItems += checkbox.Text + ", ";
        }
    }
    // Display the results
    if (selectedItems.Length > 20) // Checks if any items were added
    {
        MessageBox.Show(selectedItems.TrimEnd(' ', ','));
    }
    else
    {
        MessageBox.Show("No items were selected.");
    }
}
```

10. Radio Button:

A radio button control (also known as an option button) allows a user to select a single choice from a set of mutually exclusive options. You can create and manage radio buttons at design-time using the Visual Studio designer or dynamically through code.

Grouping radio buttons

Radio buttons are mutually exclusive by default when placed directly on a form. To create separate groups that function independently, you must place each group inside a container control, such as a `GroupBox` or `Panel`.

Example:

To create two groups of radio buttons, put the first group inside `groupBox1` and the second group inside `groupBox2`.

```
// The radio buttons inside groupBox1 are mutually exclusive.  
// The radio buttons inside groupBox2 are also mutually exclusive.  
// Selections made in one group will not affect the other.
```

- **Method 1: Drag and drop (Design-time) Using the Visual Studio Designer:**

1. In Visual Studio, create a new **Windows Forms App** project.
2. Open your form in the designer.
3. From the **Toolbox** (`View > Toolbox`), drag and drop `RadioButton` controls onto your form.
4. Optionally, drag and drop `GroupBox` controls to create separate groups.
5. Select each radio button and use the **Properties** window (`View > Properties Window`) to customize its `Text`, `Name`, and other properties.

- **Method 2: Create dynamically (Run-time) or Programmatically using C# code:**

You can also create and configure radio buttons dynamically at run-time.

```
public Form1()  
{  
    InitializeComponent();  
    CreateRadioButtons();  
}  
  
private void CreateRadioButtons()  
{  
    // Create a new radio button instance  
    RadioButton newRadioButton = new RadioButton();  
  
    // Set its properties  
    newRadioButton.Text = "Dynamic Option";  
    newRadioButton.Location = new Point(20, 150);  
}
```

```
newRadioButton.Checked = true; // Set as default selected

// Add the control to the form
this.Controls.Add(newRadioButton);
}
```

Checking the selected radio button

You can determine which radio button in a group is selected by checking its `Checked` property. The `Checked` property returns `true` if the radio button is selected and `false` otherwise.

Example: Check on a button click

Add a `Button` control to your form and implement its `Click` event handler.

```
private void submitButton_Click(object sender, EventArgs e)
{
    if (radioButton1.Checked)
    {
        MessageBox.Show("Radio Button 1 was selected.");
    }
    else if (radioButton2.Checked)
    {
        MessageBox.Show("Radio Button 2 was selected.");
    }
    else if (radioButton3.Checked)
    {
        MessageBox.Show("Radio Button 3 was selected.");
    }
    else
    {
        MessageBox.Show("No option was selected.");
    }
}

Use code with caution.
```

Responding to a selection change

To execute code immediately when a user clicks a radio button, use the `CheckedChanged` event. This event is fired twice during a selection change—once for the previously selected button becoming unchecked and again for the new one becoming checked. To handle this, check the `Checked` state of the `sender` object.

```
private void radioButton_CheckedChanged(object sender, EventArgs e)
{
    // Cast the sender object to a RadioButton
    RadioButton rb = sender as RadioButton;

    // Proceed only if the button is now checked
    if (rb != null && rb.Checked)
    {
        MessageBox.Show($"You selected: {rb.Text}");
    }
}
```

You can use the same event handler for multiple radio buttons by wiring them up in the designer.

1. Select all the radio buttons in a group.
2. In the **Properties** window, go to the **Events** tab (the lightning bolt icon).
3. Find the `CheckedChanged` event and double-click its entry to create and link a single event handler for all selected controls.

11. ListBox:

A `ListBox` is a Windows Forms control in C# that displays a list of items from which a user can select one or more entries. A scrollbar is automatically added if the number of items exceeds the display area. Items can be added and removed dynamically, and you can programmatically control which items are selected or retrieve the selected item(s). It is part of the `System.Windows.Forms` namespace.

ou can create and manage `ListBox` controls at design time using the Visual Studio designer or dynamically at runtime using C# code.

How to use a `ListBox`

1. Add items to a `ListBox`

You can populate a `ListBox` in several ways, including adding items individually, adding an array of items, or binding to a data source.

Add items in the Visual Studio designer

1. Select the `ListBox` on your form.
2. In the Properties window, click the ellipsis (`...`) next to the **Items** property.
3. In the **String Collection Editor**, type each item on a new line and click **OK**.

Add items dynamically at runtime

```
// Add items individually
listBox1.Items.Add("Apple");
listBox1.Items.Add("Banana");
listBox1.Items.Add("Cherry");

// Add items from an array
string[] fruits = { "Grape", "Lemon", "Orange" };
listBox1.Items.AddRange(fruits);

// Add items from a data source
// For example, from an ArrayList
ArrayList authors = new ArrayList();
authors.Add("Mahesh Chand");
authors.Add("Mike Gold");
listBox1.Items.Clear();
listBox1.DataSource = authors;
```

2. Get selected items

To retrieve the user's selection, you can access the `SelectedItem` or `SelectedItems` properties.

Single selection mode (default)

```
private void getSelectedItemButton_Click(object sender, EventArgs e)
{
    // Check if an item is selected
    if (listBox1.SelectedItem != null)
    {
        string selectedItemText = listBox1.SelectedItem.ToString();
        MessageBox.Show("You selected: " + selectedItemText);
    }
}
```

Multi-select

mode

To enable multiple selections, first set the `SelectionMode` property in the designer or code.

```
// Set selection mode to allow multiple items
listBox1.SelectionMode = SelectionMode.MultiSimple;

private void getSelectedItemsButton_Click(object sender, EventArgs e)
{
    // Loop through the selected items and display them
    foreach (object item in listBox1.SelectedItems)
    {
        MessageBox.Show("Selected: " + item.ToString());
    }
}
```

3. Handle key properties and events

Key properties

- `Items` : Gets the collection of items in the list box.
- `SelectedItem` : Gets the single currently selected item.
- `SelectedIndex` : Gets the index of the single currently selected item (returns -1 if none is selected).
- `SelectedItems` : Gets a collection of all selected items in multi-select mode.
- `SelectedIndices` : Gets a collection of the indices of all selected items.

- **SelectionMode** : Determines if the user can select one, multiple, or no items. Options are `None` , `One` , `MultiSimple` , and `MultiExtended` .
- **Sorted** : A boolean value that, when set to `true` , sorts the list items alphabetically.

Key events

- **SelectedIndexChanged** : Occurs when the user selects a different item in the list.
- **MouseDoubleClick** : Occurs when the user double-clicks on an item.

Example using an event handler

This example uses the `SelectedIndexChanged` event to display the newly selected item in a label.

1. Add a `ListBox` and a `Label` to your form.
2. Double-click the `ListBox` in the designer to create the `SelectedIndexChanged` event handler.
3. Add the following code to the event handler:

```
private void listBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    // Ensure an item is actually selected
    if (listBox1.SelectedItem != null)
    {
        // Display the text of the selected item in a Label
        label1.Text = "Selected: " + listBox1.SelectedItem.ToString();
    }
    else
    {
        label1.Text = "No item selected";
    }
}
```

Key Features and Usage:

- **Displaying Items:**

- Items can be added to the ListBox at design time using the Items property in the Properties window of Visual Studio.
- Items can also be added programmatically at runtime using the Add() method of the Items collection.

```
// Adding items at runtime
listBox1.Items.Add("Item 1");
listBox1.Items.Add("Item 2");
```

- **Selection Modes:**

- The SelectionMode property determines how many items can be selected.
- SelectionMode.One: Allows only a single item to be selected.
- SelectionMode.MultiSimple: Allows multiple items to be selected or deselected by clicking.
- SelectionMode.MultiExtended: Allows multiple items to be selected using Ctrl+click for individual items and Shift+click for a range of items.

- **Retrieving Selected Items:**

- For single selection, use the SelectedItem property to get the selected item or SelectedIndex to get its index.
- For multiple selections, use the SelectedItems collection to get all selected items or SelectedIndices to get their indices.

```
// Retrieving selected item (single selection)
string selectedItem = listBox1.SelectedItem.ToString();
```

```
// Retrieving selected items (multiple selection)
foreach (string item in listBox1.SelectedItems)
{
    // Process each selected item
}
```

- **Multi-column Display:**

- The MultiColumn property, when set to true, enables the display of items in columns, which can be useful for displaying a large number of items without excessive vertical scrolling.

- **Owner-Drawn ListBoxes:**

- For advanced customization, the `DrawMode` property can be set to `OwnerDrawFixed` or `OwnerDrawVariable`. This allows you to handle the `MeasureItem` and `DrawItem` events to custom draw items, enabling features like variable-height items, images, or different fonts/colors for individual items.

- **Adding/Deleting Items Programmatically:**

- `Add()`: Adds an item to the list.
- `Insert()`: Inserts an item at a specific index.
- `Remove()`: Removes a specific item.
- `RemoveAt()`: Removes an item at a specific index.
- `Clear()`: Removes all items from the list.

12. Checked List Box:

The `CheckedListBox` control in C# Windows Forms applications extends the functionality of a standard `ListBox` by displaying a checkbox next to each item in the list. This allows users to select multiple items by checking their corresponding boxes.

Key Features and Usage:

- **Adding Items:**

- **Design Time:** Items can be added directly through the String Collection Editor in the Properties window of the `CheckedListBox` control in Visual Studio.

- **Run Time:** Items can be added programmatically using the `Items.Add()` method.

Code

```
checkedListBox1.Items.Add("Item 1");  
checkedListBox1.Items.Add("Item 2", true); // Adds "Item 2" and checks it initially
```

- **Retrieving Checked Items:**

- The `CheckedItems` property provides a collection of the items that are currently checked.

- The `CheckedIndices` property provides a collection of the indices of the checked items.

```
foreach (string item in checkedListBox1.CheckedItems)
```

```
{  
    MessageBox.Show($"Checked item: {item}");  
}
```

```

    }

    foreach (int index in checkedListBox1.CheckedIndices)
    {
        MessageBox.Show($"Checked index: {index}, Value: {checkedListBox1.Items[index]}");
    }

```

Item States:

Each item in a `CheckedListBox` can have one of three states:

- **Checked:** The checkbox is checked.
- **Unchecked:** The checkbox is unchecked.
- **Indeterminate:** The checkbox displays an intermediate state (often a grayed-out checkmark), indicating a mixed or partial selection.

Events:

- **ItemCheck event:** Fired before an item's check state changes, allowing for validation or custom logic.
- **SelectedIndexChanged event:** Fired when the selected item (highlighted item) in the list changes. Note that selection and checking are distinct.

Properties:

- **ClickOnCheck:** If set to true, clicking an item in the list immediately toggles its checkbox state. If false, a second click is required to toggle the checkbox after selecting the item.
- **DisplayMember:** Specifies the property of an object to display in the list when binding to a data source.
- **ValueMember:** Specifies the property of an object to use as the actual value of an item when binding to a data source.
- **Items:** An `ObjectCollection` that holds the items displayed in the list. Items can be added at design time via the Visual Studio Properties window or at run time programmatically.
- **CheckedItems:** A collection that provides access to all the items that are currently checked.
- **CheckedIndices:** A collection that provides the zero-based indexes of the checked items.

- **CheckOnClick** : A **Boolean** property. If **true** , a single click on an item will both select it and toggle its checkbox state. If **false** , a user must click once to select an item and a second time to check it.
- **Sorted** : A **Boolean** property that, when **true** , sorts the list items alphabetically.
- **ThreeDCheckBoxes** : A **Boolean** property that changes the appearance of the checkboxes from a flat style to a 3D style.
- **CheckState** : The individual items support three states: **Checked** , **Unchecked** , and **Indeterminate** .

The **CheckedListBox** control is useful for scenarios requiring users to select multiple options from a predefined list, such as selecting features in an application or choosing preferences in a settings dialog.

Common tasks and code examples

Add items to a **CheckedListBox**

You can add items either individually or from an array or collection.

```
// Add an individual item
checkedListBox1.Items.Add("Option 1");

// Add an item and set its initial checked state
checkedListBox1.Items.Add("Option 2", true); // Set as checked

// Add multiple items from an array
string[] items = { "Option 3", "Option 4", "Option 5" };
checkedListBox1.Items.AddRange(items);
Use code with caution.
```

Get all checked items

The most common task is to process the items the user has selected. You can iterate through the **CheckedItems** collection.

```
// Iterate through the CheckedItems collection
```

```
foreach (object item in checkedListBox1.CheckedItems)
{
    // Process the checked item (e.g., display in a MessageBox)
    MessageBox.Show(item.ToString());
}
Use code with caution.
```

Get checked items by index

If you need both the index and the item text, you can iterate through the main `Items` collection and check the state of each item.

```
for (int i = 0; i < checkedListBox1.Items.Count; i++)
{
    if (checkedListBox1.GetItemChecked(i))
    {
        string itemText = checkedListBox1.Items[i].ToString();
        MessageBox.Show($"Item at index {i} is checked: {itemText}");
    }
}
```

Respond to changes (event handling)

The `ItemCheck` event is the most useful for tracking when a user checks or unchecks an item. This event fires *before* the item's checked state is changed, so you must use the event arguments to get the new state.

```
// Event handler for the ItemCheck event
private void checkedListBox1_ItemCheck(object sender, ItemCheckEventArgs e)
{
    // The e.NewValue property contains the new CheckState
    string itemText = checkedListBox1.Items[e.Index].ToString();
    if (e.NewValue == CheckState.Checked)
    {
        MessageBox.Show($"Item {itemText} is about to be checked.");
    }
    else if (e.NewValue == CheckState.Unchecked)
    {
        MessageBox.Show($"Item {itemText} is about to be unchecked.");
    }
}
```

```
}
```

Set all items to checked or unchecked

You can loop through all the items and use the `SetItemChecked` method to programmatically change their checked state.

```
// Check all items
for (int i = 0; i < checkedListBox1.Items.Count; i++)
{
    checkedListBox1.SetItemChecked(i, true);
}

// Uncheck all items
for (int i = 0; i < checkedListBox1.Items.Count; i++)
{
    checkedListBox1.SetItemChecked(i, false);
}
```

13. ComboBox:

The `ComboBox` control in C# Windows Forms applications combines the functionality of a `TextBox` and a `ListBox`. It allows users to either select an item from a predefined list or, depending on its `DropDownStyle` property, type in a new value. The `ComboBox` is part of the `System.Windows.Forms` namespace.

Key Features and Usage:

- **Adding Items:**

Items can be added to a `ComboBox` at design time or programmatically at runtime.

- **Design-Time:**

In the Visual Studio designer, you can add items directly by following these steps:

1. Add a `ComboBox` control from the **Toolbox** to your form.
2. In the **Properties** window, find the `Items` property and click the ellipsis (`...`) button.
3. In the `String Collection Editor`, type each item on a new line.

- **Run-Time (Programmatically):** You can add, insert, or clear items from the `Items` collection using code.

```
// Add a single item
comboBox1.Items.Add("Item 1");

// Add an array of items
string[] items = {"Item 2", "Item 3", "Item 4"};
comboBox1.Items.AddRange(items);

// Insert an item at a specific index
comboBox1.Items.Insert(0, "Inserted Item");

// Clear all items
comboBox1.Items.Clear();
```

- **Retrieving the selected item**

You can use several properties to retrieve the value of the currently selected item.

```
// Get the selected item as a string (useful for items added directly as strings)
string selectedText = comboBox1.Text;

// Get the entire selected object
object selectedObject = comboBox1.SelectedItem;

// Get the index of the selected item (zero-based)
int selectedIndex = comboBox1.SelectedIndex;

// Example of displaying the selected information
private void button1_Click(object sender, EventArgs e)
{
    // Check if an item is selected
    if (comboBox1.SelectedIndex != -1)
    {
        MessageBox.Show($"Selected Text: {comboBox1.Text}\nSelected Index: {comboBox1.SelectedIndex}");
    }
}
```

- **Handling selected item changes**

To respond to a user selecting a new item, use the `SelectedIndexChanged` event.

```
// In the constructor or Load event, subscribe to the event
public Form1()
{
    InitializeComponent();
    comboBox1.SelectedIndexChanged += new
    EventHandler(ComboBox1_SelectedIndexChanged);
}

// Event handler for when the selected index changes
private void ComboBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    // Your code here, such as updating other controls
    if (comboBox1.SelectedItem != null)
    {
        MessageBox.Show("You selected: " + comboBox1.SelectedItem.ToString());
    }
}
```

- **Populating with Data (Data Binding).**

For more complex scenarios, you can bind a `ComboBox` to a data source, such as a `DataTable` or a list of objects.

```
// Example: Binding to a list of custom objects
public class ComboboxItem
{
    public string Text { get; set; }
    public object Value { get; set; }

    public override string ToString()
    {
        return Text;
    }
}

// In your form's load event
```

```

private void Form1_Load(object sender, EventArgs e)
{
    // Create a list of objects to bind to
    var items = new List<ComboBoxItem>();
    items.Add(new ComboBoxItem { Text = "User 1", Value = 1 });
    items.Add(new ComboBoxItem { Text = "User 2", Value = 2 });

    // Set the data source and display/value members
    comboBox1.DataSource = items;
    comboBox1.DisplayMember = "Text"; // Property to display
    comboBox1.ValueMember = "Value"; // Property to use as the value
}

// Example of retrieving the selected value from a data-bound combo box
private void button2_Click(object sender, EventArgs e)
{
    if (comboBox1.SelectedValue != null)
    {
        MessageBox.Show($"Selected Value: {comboBox1.SelectedValue.ToString()}");
    }
}

```

- **Getting Selected Value:**

- comboBox1.SelectedItem: Returns the selected item as an object.
- comboBox1.SelectedValue: Returns the value of the ValueMember for the selected item (if data-bound).
- comboBox1.Text: Returns the text currently displayed in the ComboBox.
- comboBox1.SelectedIndex: Returns the zero-based index of the selected item

- **Events:**

- SelectedIndexChanged: Fires when the selected item in the drop-down list changes.
- TextUpdate: Fires when the text portion of the ComboBox is updated.
- SelectionChangeCommitted: Fires after the user has made a selection from the drop-down list and the selection has been committed.

- **DropDownStyle Property:**
 - DropDown: Default. Allows users to type in the text box or select from the list.
 - DropDownList: Users can only select from the list; they cannot type in the text box.
 - Simple: The list is always visible, and users can type in the text box.

14. Month Calendar:

In C# Windows Forms applications, the primary calendar control available is the MonthCalendar control. This control provides a graphical interface for users to select a date or a range of dates. It is useful when you need to select one or more dates from a calendar grid.

Adding a MonthCalendar Control:

- **Using the Designer:**
 - In Visual Studio, open a WinForms project.
 - Drag and drop a MonthCalendar control from the Toolbox onto your form.
 - Access the control's properties and events through the Properties window.
- **Using Code:**

```
MonthCalendar monthCalendar1 = new MonthCalendar();
this.Controls.Add(monthCalendar1);
```

Key Properties of the MonthCalendar Control:

- **SelectionMode:** Determines whether a single date or a range of dates can be selected.
- **SelectionStart and SelectionEnd:** Get or set the start and end dates of the current selection.
- **AnnuallyBoldedDates and MonthlyBoldedDates:** Similar to BoldedDates, but for recurring annual or monthly dates.
- **SelectionRange:** Gets or sets the currently selected range of dates.
- **MinDate and MaxDate:** Set the minimum and maximum dates that the user can select.
- **BoldedDates:** Sets specific dates to be displayed in bold.
- **DateSelected event:** Fires when the user selects a new date.

Appearance

Properties: ForeColor, BackColor, Font, TitleBackColor, TitleForeColor, TrailingForeColor for customizing the control's visual style.

Handling Events:

The MonthCalendar control provides events to respond to user interactions, such as:

- DateChanged: Occurs when the selected date or range of dates changes.
- DateSelected: Occurs when the user selects a date (or range).

Example:

To display the selected date in a `Label` control, double-click the `MonthCalendar` in the designer to create the `DateSelected` event handler.

```
private void monthCalendar1_DateSelected(object sender, DateRangeEventArgs e)
{
    // Display the selected date range in a label
    label1.Text = "Start Date: " + e.Start.ToShortDateString() +
        "\nEnd Date: " + e.End.ToShortDateString();
}
```

Example:

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();

        // Add a MonthCalendar control to the form
        MonthCalendar monthCalendar1 = new MonthCalendar();
        monthCalendar1.Location = new Point(10, 10);

        // Allow single date selection
        monthCalendar1.SelectionMode = MonthCalendarSelectionMode.One;

        // Limit selection to one year from today
        monthCalendar1.MaxDate = DateTime.Today.AddYears(1);
        this.Controls.Add(monthCalendar1);
    }
}
```

```

// Handle the DateSelected event
monthCalendar1.DateSelected += MonthCalendar1_DateSelected;
}

private void MonthCalendar1_DateSelected(object sender, DateRangeEventArgs e)
{
    MessageBox.Show($"Selected Date: {e.Start.ToShortDateString()}");
}
}

```

15. DateTime Picker:

The DateTimePicker control in C# Windows Forms applications provides a user-friendly way to select and display date and time values. It is found in the System.Windows.Forms namespace.

Key Features and Usage:

- **Adding the control in the Visual Studio designer**
 - Open your form in the Visual Studio designer.
 - Open the **Toolbox** (View > Toolbox or **Ctrl+Alt+X**).
 - Drag the **DateTimePicker** control from the "Common Controls" group onto your form.
 - Right-click the control and select **Properties** to customize its appearance and behavior.
- **Displaying Date or Time:**
 - By default, it displays a calendar for date selection.
 - To display time, set the ShowUpDown property to true and the Format property to Time. This will show up/down arrows for adjusting the time.
- **Setting and Retrieving Values:**
 - The Value property holds the currently selected DateTime object. You can set it programmatically or retrieve the user's selection.
 - The Text property returns the entire value as formatted in the control.
- **Formatting Options:**

- The Format property allows you to choose from predefined formats: Long, Short, Time, or Custom.
- If Format is set to Custom, you can define your own display format using the CustomFormat property with format specifiers (e.g., "MM/dd/yyyy hh:mm:ss").
- **Date Range Restrictions:**
 - The MinDate and MaxDate properties allow you to set the minimum and maximum selectable dates, restricting user input to a specific range.
- **Optional Checkbox:**
 - Setting the ShowCheckBox property to true displays a checkbox next to the selected date. When checked, the value can be updated; when unchecked, it appears unavailable.

Retrieving the selected date and time

Use the `Value` property of the `DateTimePicker` to get the selected `DateTime` object.

```
private void button1_Click(object sender, EventArgs e)
{
    // Retrieve the selected DateTime value
    DateTime selectedDate = dateTimePicker1.Value;

    // Display the value in a message box
    MessageBox.Show("The selected date and time is: " + selectedDate.ToString());
}
```

Customizing the display format

You can change the way the date and time are displayed by setting the `Format` and `CustomFormat` properties.

```
// Display a short date format (e.g., "9/29/2025")
dateTimePicker1.Format = DateTimePickerFormat.Short;

// Display a time-only picker
dateTimePicker1.Format = DateTimePickerFormat.Time;
dateTimePicker1.ShowUpDown = true; // Use up/down buttons for time
```

```
// Display a custom format (e.g., "Monday, September 29, 2025")  
dateTimePicker1.Format = DateTimePickerFormat.Custom;  
dateTimePicker1.CustomFormat = "dddd, MMMM dd, yyyy";
```

Other key properties

- `MinDate` and `MaxDate` : Set the minimum and maximum selectable dates.
- `ShowCheckBox` : Adds a checkbox that, when unchecked, makes the value appear unavailable.
- `ShowUpDown` : Replaces the calendar drop-down with up/down buttons for adjusting the date or time.

Example of Custom Formatting:

```
// Assuming you have a DateTimePicker control named dateTimePicker1  
dateTimePicker1.Format = DateTimePickerFormat.Custom;  
dateTimePicker1.CustomFormat = "dddd, MMMM dd, yyyy hh:mm:ss tt";  
  
// Example custom format  
  
dateTimePicker1.Format = DateTimePickerFormat.Custom;  
dateTimePicker1.CustomFormat = "yyyy/MM/dd hh:mm:ss";
```

Similarly, if you want the date to be appeared in this format i.e date/month/year. You can use the following code.

```
dateTimePicker1.Format = DateTimePickerFormat.Custom;  
dateTimePicker1.CustomFormat = "dd/MM/yyyy";
```

Example of Handling Value Changes:

```
// Event handler for when the DateTimePicker value changes  
private void dateTimePicker1_ValueChanged(object sender, EventArgs e)  
{  
    // Display the selected date and time in a Label  
    label1.Text = "Selected Date and Time: " + dateTimePicker1.Value.ToString();  
}
```

16. TreeView:

The TreeView control in C# Windows Forms applications allows the display of hierarchical data in a tree-like structure, with nodes that can be expanded and collapsed.

Key Concepts and Usage:

- **Adding the TreeView Control:**

- Drag and drop the TreeView control from the Toolbox onto your Windows Form in Visual Studio.

- **Adding Nodes Programmatically:**

- Nodes are represented by TreeNode objects.
- You can add top-level nodes directly to the TreeView.Nodes collection.
- Child nodes are added to the Nodes collection of their parent TreeNode.

Code

```
// Create a root node
```

```
TreeNode rootNode = new TreeNode("Root Item");  
treeView1.Nodes.Add(rootNode);
```

```
// Add a child node to the root
```

```
TreeNode childNode1 = new TreeNode("Child Item 1");  
rootNode.Nodes.Add(childNode1);
```

```
// Add another child to the root
```

```
TreeNode childNode2 = new TreeNode("Child Item 2");  
rootNode.Nodes.Add(childNode2);
```

```
// Add a grandchild to childNode1
```

```
TreeNode grandChildNode = new TreeNode("Grandchild Item");  
childNode1.Nodes.Add(grandChildNode);
```

- **Populating from Data Sources:**

- You can dynamically populate the TreeView from various data sources like databases, XML files, or custom object collections.
- This often involves iterating through the data and creating TreeNode objects accordingly, establishing parent-child relationships.

- **Handling Node Selection:**

- The AfterSelect event is commonly used to respond when a user selects a node.

- The `SelectedNode` property of the `TreeView` control provides access to the currently selected `TreeNode` object.

```
private void treeView1_AfterSelect(object sender, TreeViewEventArgs e)
{
    // Get the selected node
    TreeNode selectedNode = e.Node;

    // Perform actions based on the selected node
    MessageBox.Show($"Selected: {selectedNode.Text}");
}
```

- **Node Properties:**

- **Text:** The display text of the node.
- **Tag:** An object property that can store custom data associated with the node (e.g., a database ID).
- **Nodes:** The collection of child `TreeNode` objects.
- **Parent:** The parent `TreeNode` of the current node.

- **TreeView Properties:**

- **ShowLines:** Determines whether lines connecting nodes are displayed.
- **CheckBoxes:** If set to true, displays checkboxes next to each node.
- **ImageList:** Allows associating images with nodes using an `ImageList` component.
- **ImageIndex:** The default image index for nodes from the `ImageList`.
- **SelectedImageIndex:** The image index for the selected node.

Example of Adding and Removing Nodes:

```
// Add a new node to the selected node (or as a root if no node is selected)
private void AddNodeButton_Click(object sender, EventArgs e)
{
    string newNodeText = "New Item"; // Or get from a TextBox
    TreeNode newNode = new TreeNode(newNodeText);

    if (treeView1.SelectedNode != null)
    {
        treeView1.SelectedNode.Nodes.Add(newNode);
        treeView1.SelectedNode.Expand(); // Expand the parent to show the new node
    }
    else
    {

```

```

        treeView1.Nodes.Add(newNode);
    }
}

// Remove the selected node
private void RemoveNodeButton_Click(object sender, EventArgs e)
{
    if (treeView1.SelectedNode != null)
    {
        treeView1.SelectedNode.Remove();
    }
    else
    {
        MessageBox.Show("Please select a node to remove.");
    }
}

```

17. ListView:

The ListView control in C# Windows Forms applications is used to display a collection of items, often with associated icons and sub-items, in various views.

Adding a ListView Control:

- **Design-time:**

Drag the ListView control from the Toolbox onto your Windows Form in Visual Studio.

- **Run-time:**

Create an instance of ListView in your C# code and add it to the form's controls collection.

```

ListView myListView = new ListView();
myListView.Location = new System.Drawing.Point(10, 10);
myListView.Size = new System.Drawing.Size(200, 150);
this.Controls.Add(myListView);

```

Configuring the ListView:

- **View Property:** Set the View property to control how items are displayed:
 - Details: Displays items and sub-items in a grid with column headers.
 - LargeIcon: Displays large icons with item text below.
 - SmallIcon: Displays small icons with item text to the right.

- List: Displays small icons with item text in a vertical list.
- Tile: Displays items as tiles (limited availability).
- **Columns (for Details View):** Add ColumnHeader objects to the Columns collection to define the columns.


```
myListView.View = View.Details;
myListView.Columns.Add("Name", 100);
myListView.Columns.Add("Age", 50);
```

Adding Items to the ListView:

- Create ListViewItem objects: Each item in the ListView is represented by a ListViewItem object.
- **Add Sub-items (for Details View):** If using Details view, add sub-items to the ListViewItem to populate the columns.


```
ListViewItem item1 = new ListViewItem("John");
item1.SubItems.Add("30"); // Age sub-item
myListView.Items.Add(item1);

ListViewItem item2 = new ListViewItem("Jane");
item2.SubItems.Add("25");
myListView.Items.Add(item2);
```

Handling Events:

The ListView control provides various events for user interaction, such as ItemSelectionChanged, ItemActivate, and MouseClick, allowing you to respond to user actions.

18. Timer:

To implement a timer control in a C# Windows Forms application, follow these steps:

- **Create a Windows Forms Application:**

Open Visual Studio and create a new Windows Forms App (.NET Framework) project.

- **Add Controls to the Form:**

- Drag and drop a Timer control from the Toolbox onto your form. This adds a Timer component (e.g., timer1) to the component tray at the bottom of the designer.

- Add any other controls you need for your application's functionality, such as Button controls to start and stop the timer, and a Label or TextBox to display timer-related information.
- **Configure the Timer Properties:**
 - Select the Timer control in the component tray.
 - In the Properties window, set the Interval property. This value is in milliseconds. For example, 1000 milliseconds equals 1 second.
 - Initially, you can set the Enabled property to false if you want the timer to start only when a specific action (like clicking a button) occurs.
 - Handle the Tick Event:
 - In the Properties window for the Timer control, click the "Events" button (lightning bolt icon).
 - Double-click the Tick event to create an event handler method (e.g., timer1_Tick).
 - Write the code you want to execute repeatedly at the specified Interval within this timer1_Tick method. For instance, you might update a Label with the current time or perform a specific task.
- **Start and Stop the Timer (Optional):**
 - If you added Button controls for starting and stopping the timer, double-click them in the designer to create their Click event handlers.
 - In the "Start" button's Click event handler, call timer1.Start() to begin the timer.
 - In the "Stop" button's Click event handler, call timer1.Stop() to halt the timer.

Example Code Snippet:

```
using System;
using System.Windows.Forms;

namespace TimerApp
{
    public partial class Form1 : Form
    {
        private int count = 0;
```

```

public Form1()
{
    InitializeComponent();
    // Optional: Set initial properties in code
    timer1.Interval = 1000; // 1 second
    timer1.Enabled = false; // Start disabled
}

private void startButton_Click(object sender, EventArgs e)
{
    timer1.Start();
    count = 0; // Reset count on start
    displayLabel.Text = "Timer started!";
}

private void stopButton_Click(object sender, EventArgs e)
{
    timer1.Stop();
    displayLabel.Text = "Timer stopped.";
}

private void timer1_Tick(object sender, EventArgs e)
{
    count++;
    displayLabel.Text = $"Count: {count}";
    // Add other actions to perform at each tick here
    // For example, if (count >= 10) { timer1.Stop(); }
}
}

```

This example demonstrates how to set up a basic timer that updates a label with a count every second. You can adapt the `timer1_Tick` event handler to perform any desired actions at regular intervals within your Windows Forms application.

19. TrackBar:

The `TrackBar` control in a C# Windows Forms application provides a visual slider for users to select a value within a specified range.

Adding a `TrackBar` Control:

- **Design-time:**

Drag and drop a TrackBar control from the Toolbox onto your Windows Form in Visual Studio.

- **Run-time (code):**

Create an instance of the TrackBar class in your C# code and add it to the form's controls collection.

Key Properties:

- Minimum: Sets the minimum value of the slider.
- Maximum: Sets the maximum value of the slider.
- Value: Gets or sets the current position of the slider.
- Orientation: Determines whether the track bar is horizontal or vertical.
- LargeChange: Specifies the increment for large changes (e.g., clicking the track bar itself or using Page Up/Page Down keys).
- SmallChange: Specifies the increment for small changes (e.g., using arrow keys).
- TickFrequency: Sets the interval between tick marks.
- TickStyle: Defines how tick marks are displayed (e.g., BottomRight, TopLeft, Both, None).

Handling the Scroll Event:

The TrackBar control exposes a Scroll event that fires whenever the slider's position changes. You can handle this event to respond to user interactions and update other parts of your application based on the TrackBar's Value.

Example Code (Design-time setup):

Assuming you have a TrackBar named trackBar1 and a Label named label1 on your form:

// In the constructor or Load event of your Form

```
public Form1()
{
    InitializeComponent();

    // Set initial properties for the TrackBar
    trackBar1.Minimum = 0;
    trackBar1.Maximum = 100;
    trackBar1.Value = 50; // Initial value
    trackBar1.TickFrequency = 10;
    trackBar1.LargeChange = 5;
    trackBar1.SmallChange = 1;
```

```

// Display initial value in the label
label1.Text = $"Value: {trackBar1.Value}";
}

// Event handler for the TrackBar's Scroll event
private void trackBar1_Scroll(object sender, EventArgs e)
{
    // Update the label with the current TrackBar value
    label1.Text = $"Value: {trackBar1.Value}";

    // You can also use trackBar1.Value to control other elements,
    // for example, changing the size of a control or adjusting a setting.
    // Example: button1.Width = trackBar1.Value * 2;
}

```

20. ProgressBar:

To implement a ProgressBar control in a C# Windows Forms application, follow these steps:

- **Add a ProgressBar Control:**

- Open your Windows Forms project in Visual Studio.
- Drag a ProgressBar control from the Toolbox onto your Form.
- You can also add it programmatically:
Code

```

ProgressBar progressBar1 = new ProgressBar();
this.Controls.Add(progressBar1);

```

- **Configure ProgressBar Properties:**

- Minimum: Sets the minimum value of the progress bar (e.g., 0).
- Maximum: Sets the maximum value, representing 100% completion (e.g., 100).
- Value: Sets the current progress value. This will be updated as the task progresses.
- Step: Defines the amount by which the Value property is incremented when PerformStep() is called.
- Style: Determines how the progress is displayed:
- Continuous: A solid bar.
- Blocks: A series of blocks.
- Marquee: A scrolling block, indicating an ongoing but indeterminate process.

- Update the ProgressBar's Value:
 - **Directly setting Value:**
 progressBar1.Value = 50; // Sets progress to 50%
 - **Using PerformStep().**
 progressBar1.Step = 10; // Set the step increment
 progressBar1.PerformStep(); // Increments the value by 10
 - **Using Increment().**
 progressBar1.Increment(25); // Increments the value by 25

- **Example Scenario (using a Button and a long-running task):**

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
        progressBar1.Minimum = 0;
        progressBar1.Maximum = 100;
        progressBar1.Value = 0;
    }

    private void startButton_Click(object sender, EventArgs e)
    {
        // Simulate a long-running task
        for (int i = 0; i <= 100; i++)
        {
            // Update the progress bar
            progressBar1.Value = i;

            // Simulate work
            System.Threading.Thread.Sleep(50); // Pause for 50 milliseconds
        }

        MessageBox.Show("Task Completed!");
    }
}
```

Note: For actual long-running operations that shouldn't block the UI thread, consider using BackgroundWorker or async/await with Task.Run() to update the ProgressBar from a different thread while ensuring UI updates occur on the main thread (using Invoke or BeginInvoke if necessary, or the ProgressChanged event of BackgroundWorker).

21. ImageList:

The ImageList control in a C# Windows Forms application serves as a container for a collection of images. It is not typically displayed directly on a form but is used by other controls, such as ListView, TreeView, ToolBar, Button, TabControl, CheckBox, RadioButton, and Label, to provide visual elements.

Key features and usage of ImageList:

- **Adding Images:**

- **Designer:** You can add an ImageList component to your form from the Toolbox. Then, using the Properties window or the smart tag, you can add images from various file sources.
- **Programmatically:** Images can be added dynamically at runtime using the Images.Add() method. You can load images from files using Image.FromFile() and then add them to the ImageList.

```
// Example of adding an image programmatically  
ImageList myImageList = new ImageList();  
Image myImage = Image.FromFile("path/to/your/image.png");  
myImageList.Images.Add(myImage);
```

- **Referencing Images:**

- Images within an ImageList are indexed starting from zero. You can refer to them by their index or, if specified, by a key (name).
- Other controls that use an ImageList (like ListView or TreeView) will have properties like ImageList, SmallImageList, or LargeImageList that you set to an instance of your ImageList component. Then, you can specify the ImageIndex or ImageKey property of individual items within those controls to display a specific image from the ImageList.

- **Properties:**

- **ImageSize:** Determines the size of the images stored in the ImageList. All images will be scaled to this size.
- **ColorDepth:** Specifies the color depth of the images.
- **TransparentColor:** Sets a color to be treated as transparent within the images.

Example Scenario:

Imagine creating a ListView that displays a list of files with icons. You would:

- Add an ImageList component to your form.

- Add various file type icons (e.g., for documents, images, folders) to the ImageList.
- Set the SmallImageList or LargeImageList property of your ListView to your ImageList component.
- When adding items to the ListView, set the ImageIndex property of each ListViewItem to the index of the corresponding icon in the ImageList.
By using the ImageList control, you efficiently manage and reuse image resources across multiple controls in your Windows Forms application.

22. HScrollBar:

The HScrollBar control in a C# Windows Forms application enables horizontal scrolling functionality for content that exceeds the visible area of a control or a form.

Creating an HScrollBar:

- **Design-time:** Drag and drop an HScrollBar control from the Visual Studio Toolbox onto your form.
- **Run-time (Dynamically):**

```
HScrollBar hScroller = new HScrollBar();
hScroller.Location = new Point(10, 10); // Set position
hScroller.Width = 200; // Set width
this.Controls.Add(hScroller); // Add to the form's controls
```

Key Properties:

- **Minimum:** The lowest value the scroll bar can represent (leftmost position).
- **Maximum:** The highest value the scroll bar can represent. Note that the Value when the thumb is at the far right is Maximum - LargeChange + 1.
- **SmallChange:** The amount the Value changes when the user clicks an arrow button.
- **LargeChange:** The amount the Value changes when the user clicks the scroll bar track on either side of the thumb.
- **Value:** The current position of the scroll thumb.

Events:

- **Scroll:** Occurs when the scroll thumb is moved or the scroll bar is clicked. This is where you implement the logic to update the displayed content based on the HScrollBar.Value.

Example Usage

To scroll content horizontally, such as an image or a panel:

- Set the Minimum and Maximum properties of the HScrollBar to reflect the range of your content.
- In the Scroll event handler, update the position of the content based on the HScrollBar.Value. For example, to scroll a PictureBox:

```
private void hScrollBar1_Scroll(object sender, ScrollEventArgs e)
{
    pictureBox1.Left = -hScrollBar1.Value; // Adjust position based on scroll value
}
```

23. VScrollBar:

Using the VScrollBar control in a C# Windows Forms application is necessary when you need to add scroll functionality to a container or custom control that doesn't have built-in scrolling. For standard controls like TextBox, ListBox, and Panel, it is often simpler to enable their built-in scrolling properties.

Option 1: The VScrollBar control

This approach gives you full programmatic control over the scrollbar and is ideal for custom-drawn controls, such as scrolling a PictureBox.

Example: Scrolling a PictureBox

This program demonstrates how to add and configure a VScrollBar to scroll a large image within a smaller PictureBox.

1. Set up the form

- Create a new C# Windows Forms project in Visual Studio.
- In the Form Designer, add a PictureBox control and a VScrollBar control.
- Set the Dock property of the VScrollBar to Right.
- Set the Size of the PictureBox to be smaller than the image you will load.
- Name your controls appropriately (e.g., pictureBox1, vScrollBar1).

2. Add the C# code

```
using System;
using System.Drawing;
using System.Windows.Forms;

public partial class Form1 : Form
{
    private Image largeImage;
    private int yOffset = 0;

    public Form1()
    {
        InitializeComponent();

        // Load the large image.
        // Replace "path_to_your_image.jpg" with the actual path.
        try
        {
            largeImage = Image.FromFile("path_to_your_image.jpg");
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Error loading image: {ex.Message}");
            return;
        }

        // Disable the standard PictureBox scrollbars.
        // The SizeMode must be Normal for custom scrolling.
        pictureBox1.SizeMode = PictureBoxSizeMode.Normal;

        // Configure the VScrollBar properties.
        vScrollBar1.Minimum = 0;
        vScrollBar1.Maximum = largeImage.Height - pictureBox1.Height;
        vScrollBar1.LargeChange = pictureBox1.Height / 10; // Clicks between thumb and arrow
        vScrollBar1.SmallChange = 10; // Arrow clicks

        // Hook up the events.
        vScrollBar1.Scroll += VScrollBar1_Scroll;
        pictureBox1.Paint += PictureBox1_Paint;
    }
}
```

```

// Handles the Scroll event of the VScrollBar.
private void VScrollBar1_Scroll(object sender, ScrollEventArgs e)
{
    yOffset = e.NewValue; // Get the new scroll position
    pictureBox1.Invalidate(); // Redraw the PictureBox
}

// Handles the Paint event of the PictureBox to draw the image section.
private void PictureBox1_Paint(object sender, PaintEventArgs e)
{
    if (largedImage != null)
    {
        // Draw a section of the large image onto the PictureBox.
        e.Graphics.DrawImage(largedImage,
            new Rectangle(0, 0, pictureBox1.Width, pictureBox1.Height),
            new Rectangle(0, yOffset, pictureBox1.Width, pictureBox1.Height),
            GraphicsUnit.Pixel);
    }
}
}

```

Option 2: The `Panel` control with `AutoScroll`

For standard layout containers, using the `Panel` control's built-in `AutoScroll` feature is much easier. When a control inside the panel is larger than the panel's visible area, scrollbars will appear automatically.

Example: Scrolling controls in a `Panel`

This example shows how to add a series of labels to a `Panel` that automatically handles vertical scrolling.

1. Set up the form

- Create a new C# Windows Forms project.
- In the Form Designer, add a `Panel` control.
- Set its `AutoScroll` property to `true`.

- Set its `FlowDirection` property to `TopDown` for vertical stacking.
- Set its `WrapContents` property to `false` to ensure items stay in a single column.
- Add a `Button` control to trigger the population of the panel.

2. Add the C# code

```
using System;
using System.Windows.Forms;

public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
        button1.Click += new EventHandler(Button1_Click);
    }

    private void Button1_Click(object sender, EventArgs e)
    {
        // Clear any existing labels.
        panel1.Controls.Clear();

        // Add 30 labels to the panel.
        for (int i = 1; i <= 30; i++)
        {
            Label label = new Label();
            label.Text = $"This is label number {i}.";
            label.AutoSize = true; // Labels will automatically expand to their content.
            panel1.Controls.Add(label);
        }
    }
}
```

24. Tab Control:

Using a `TabControl` in a C# Windows Forms application is a common way to organize related controls into multiple, selectable tabs on a single form. You can use the Visual Studio designer for

a simple, visual approach or create and manage the control programmatically for dynamic applications.

Method 1: Using the Visual Studio designer

This method is the simplest for building a `TabControl` at design time.

1. **Create a Windows Forms Project:** Open Visual Studio and create a new project of type "Windows Forms App (.NET Framework)" or "Windows Forms App (.NET)".
2. **Add a `TabControl` :**
 - Open the `Toolbox` (View > Toolbox).
 - Find the `TabControl` component and drag and drop it onto your form.
3. **Add Tab Pages:**
 - With the `TabControl` selected, click the small arrow in the top-right corner to open the `TabControl Tasks` menu.
 - Click **Add Tab** to create new tab pages. You can also remove tabs from this menu.
 - Alternatively, you can go to the `Properties` window, find the `TabPages` property, and click the ellipsis (`...`) to open the `TabPage Collection Editor` .
4. **Rename Tabs:**
 - In the `Properties` window, select a `TabPage` to edit its properties.
 - Change the `Text` property to update the title displayed on the tab header.
5. **Add controls to a tab:**
 - Click on a tab to make it the active tab in the designer.
 - Drag and drop other controls (like `TextBox` , `Button` , or `Label`) from the `Toolbox` onto the selected tab page.

Method 2: Creating a `TabControl` programmatically

This approach is useful for applications that need to add or remove tabs dynamically at runtime.

Step 1: Create the main Form

First, create a new Windows Forms application project in Visual Studio.

Step 2: Add code to the form's constructor

Open your main form's code file (`Form1.cs`). In the constructor, add the following code to create and configure the `TabControl` and its `TabPage`s .

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();

        // Create the TabControl
        TabControl dynamicTabControl = new TabControl();
        dynamicTabControl.Dock = DockStyle.Fill; // Set it to fill the form
        dynamicTabControl.Name = "DynamicTabControl";

        // Create and add the first TabPage
        TabPage tabPage1 = new TabPage("First Tab");
        tabPage1.Name = "FirstTab";
        dynamicTabControl.TabPages.Add(tabPage1);

        // Create and add the second TabPage
        TabPage tabPage2 = new TabPage("Second Tab");
        tabPage2.Name = "SecondTab";
        dynamicTabControl.TabPages.Add(tabPage2);

        // Add controls to a TabPage
        Label label1 = new Label();
        label1.Text = "This is a label on the first tab.";
        label1.Location = new Point(10, 20);
        tabPage1.Controls.Add(label1);

        // Add the TabControl to the form
        this.Controls.Add(dynamicTabControl);
    }
}
```

```
}  
}
```

Step 3: Add and remove tabs at runtime

You can add and remove tabs based on user actions, such as a button click.

First, add a `Button` to your form using the designer. Then, double-click the button to create its `Click` event handler.

```
private void addButton_Click(object sender, EventArgs e)  
{  
    // Find the TabControl by its name  
    TabControl tabControl = (TabControl)this.Controls.Find("DynamicTabControl", true)[0];  
  
    // Create a new TabPage  
    string title = $"New Tab {tabControl.TabCount + 1}";  
    TabPage newTabPage = new TabPage(title);  
  
    // Add the new TabPage to the TabControl  
    tabControl.TabPages.Add(newTabPage);  
}
```

Key properties and events

- **TabPages**: A collection that allows you to manage the individual `TabPage` objects. You can add, remove, and access specific tabs using this property.
- **SelectedTab**: Gets or sets the currently active `TabPage`.
- **SelectedIndex**: Gets or sets the zero-based index of the active tab.
- **Controls**: Each `TabPage` has its own `Controls` collection, which is where you add other UI elements like `Buttons` or `Textboxes`.
- **SelectedIndexChanged**: This event is fired when the user selects a new tab. You can use it to update the content or state of your application based on the active tab.

Section – 2: Common Dialog Control:

1. Color Dialog:

To implement a Color Dialog control in a C# Windows Forms application, you use the `ColorDialog` component. The `ColorDialog` class displays a pre-configured system dialog that allows users to select from a palette of basic colors or define custom colors.

Step-by-step example

This guide shows how to add a button that, when clicked, opens a Color Dialog and changes the background color of a `TextBox`.

1. Set up the Windows Forms project

1. Open Visual Studio and create a new **Windows Forms App (.NET Framework)** project.
2. In the Form Designer, drag the following controls from the Toolbox onto your form:
 - `Button`
 - `TextBox`
 - `ColorDialog` (The `ColorDialog` component will appear in a tray at the bottom of the designer).

2. Name the controls

1. Select the `Button` on your form. In the **Properties** window, change its `Name` to `selectColorButton` and its `Text` to `Select Color`.
2. Select the `TextBox` on your form. Change its `Name` to `outputTextBox`.

3. Add code to handle the button click

1. Double-click the **Select Color** button to create a `Click` event handler in the code-behind file (`Form1.cs`).
2. Add the following code to the `selectColorButton_Click` event:

```
private void selectColorButton_Click(object sender, EventArgs e)
{
    // Display the Color Dialog
    if (colorDialog1.ShowDialog() == DialogResult.OK)
    {
```

```
// Set the TextBox background color to the selected color
outputTextBox.BackColor = colorDialog1.Color;
}
}
```

Explanation:

- `colorDialog1.ShowDialog()` displays the color picker window. The method returns a `DialogResult` that indicates which button the user clicked to close the dialog.
- `DialogResult.OK` confirms that the user clicked the "OK" button and did not cancel the dialog.
- If the user clicked "OK", you can retrieve the selected color using the `colorDialog1.Color` property and apply it to the `BackColor` of the `outputTextBox`.

4. Run the application

Press **F5** to build and run your application. When you click the **Select Color** button, the Color Dialog will appear. After you choose a color and click **OK**, the background of the text box will update to reflect your selection.

Customizing the Color Dialog

You can modify the appearance and behavior of the `ColorDialog` by setting its properties before calling `ShowDialog()`.

Property	Description	Example
<code>AllowFullOpen</code>	Gets or sets a value indicating whether the "Define Custom Colors" button is enabled. Setting this to <code>false</code> disables the creation of custom colors.	<code>colorDialog1.AllowFullOpen = false;</code>
<code>FullOpen</code>	Gets or sets a value indicating whether the custom color controls are visible by default when the dialog opens.	<code>colorDialog1.FullOpen = true;</code>

<code>SolidColorOnly</code>	Gets or sets a value that restricts the user to selecting only solid colors, which can prevent the selection of dithered colors on older displays.	<code>colorDialog1.SolidColorOnly = true;</code>
<code>ShowHelp</code>	Gets or sets a value indicating whether a "Help" button appears on the dialog.	<code>colorDialog1.ShowHelp = true;</code>
<code>CustomColors</code>	Gets or sets an array of custom colors shown in the dialog.	<code>colorDialog1.CustomColors = new int[] { ColorTranslator.ToOle(Color.Blue), ColorTranslator.ToOle(Color.Red) };</code>

2. Folder Browser Dialog:

To use a Folder Browser Dialog in a C# Windows Forms application, you'll use the `FolderBrowserDialog` class, which allows users to select a directory. This is useful for applications that need to save files or access contents from a specific folder.

Example: Windows Forms application

Step 1: Create a new project

1. Open Visual Studio.
2. Create a new Windows Forms App (.NET Framework) or Windows Forms App (.NET) project.
3. Design your form by dragging a `Button` and a `TextBox` from the **Toolbox** onto your form.
4. Rename the button to `btnBrowse` and the text box to `txtFolderPath`. Change the button's text to "Browse".

Step 2: Implement the code

1. Double-click the "Browse" button to create an event handler for its `Click` event.
2. In the `Click` event handler, add the following C# code:

```

private void btnBrowse_Click(object sender, EventArgs e)
{
    // Create an instance of the FolderBrowserDialog.
    FolderBrowserDialog folderDlg = new FolderBrowserDialog();

    // Set the initial directory (optional).
    // This example starts the dialog at the user's Desktop.
    folderDlg.RootFolder = Environment.SpecialFolder.Desktop;

    // Set a description for the dialog (optional).
    folderDlg.Description = "Select a folder to save your files.";

    // Show the dialog and check if the user clicked OK.
    if (folderDlg.ShowDialog() == DialogResult.OK)
    {
        // Get the selected folder path from the dialog.
        string selectedPath = folderDlg.SelectedPath;

        // Display the selected path in the TextBox.
        txtFolderPath.Text = selectedPath;

        // You can now use the 'selectedPath' variable for other operations.
        MessageBox.Show("The folder selected was: " + selectedPath);
    }
}

```

Step 3: Run the application

Run the application by pressing `F5`. When you click the "Browse" button, a standard Windows folder browser dialog will appear. After you select a folder and click **OK**, the path will be displayed in the text box.

Key properties of `FolderBrowserDialog`

- `SelectedPath`: A string that gets the path of the folder selected by the user. You can read this property after `ShowDialog()` returns `DialogResult.OK`.
- `RootFolder`: An `Environment.SpecialFolder` value that specifies the root-level folder where the browsing begins.

- **Description** : A string that provides instructions to the user at the top of the dialog box.
- **ShowNewFolderButton** : A boolean that controls whether a "Make New Folder" button is displayed in the dialog. It is **true** by default.

The modern approach: **CommonOpenFileDialog**

For applications targeting .NET 5+ or with the Windows API Code Pack installed, a more modern and flexible dialog is **CommonOpenFileDialog**. It is generally preferred because it provides a more up-to-date look and supports multiple selections.

How to use **CommonOpenFileDialog**

1. Install the **Microsoft.WindowsAPICodePack.Shell** and **Microsoft.WindowsAPICodePack.Core** NuGet packages.
2. Use the following code in your button's **Click** event handler:

```
csharp
private void btnBrowse_Click(object sender, EventArgs e)
{
    var dialog = new Microsoft.WindowsAPICodePack.Dialogs.CommonOpenFileDialog();
    dialog.IsFolderPicker = true; // Essential for folder selection

    if (dialog.ShowDialog() ==
Microsoft.WindowsAPICodePack.Dialogs.CommonFileDialogResult.Ok)
    {
        txtFolderPath.Text = dialog.FileName;
    }
}
```

3. Font Dialog:

Using the **FontDialog** control in a C# Windows Forms application is a straightforward process that allows users to select a font and its attributes. You can add the control using the Visual Studio designer or by creating an instance of the **FontDialog** class in code.

Step 1: Create a Windows Forms application

1. Open Visual Studio.
2. Go to `File > New > Project`.
3. Select the `Windows Forms App` template for C# and click **Next**.
4. Give your project a name and click **Create**.

Step 2: Add controls to the form

1. From the **Toolbox** on the left, drag and drop the following controls onto your form:
 - **Label:** A control to display text. Change its `Text` property to "Sample Text".
 - **Button:** A control to launch the font dialog. Change its `Text` property to "Change Font".
 - **FontDialog:** This is a non-visual component. Dragging and dropping it will add a `fontDialog1` instance to the component tray at the bottom of the designer.

Step 3: Implement the button click event

1. Double-click the "Change Font" button on your form. This will automatically generate a `button1_Click` event handler in your code.
2. Add the following C# code inside the `button1_Click` method:

```
private void button1_Click(object sender, EventArgs e)
{
    // Show the font dialog and check if the user clicked "OK"
    if (fontDialog1.ShowDialog() == DialogResult.OK)
    {
        // Set the font of the label to the font selected in the dialog
        label1.Font = fontDialog1.Font;
    }
}
```

Step 4: Run the application

1. Press **F5** or click the **Start** button in Visual Studio.

2. Click the **Change Font** button to open the font dialog.
3. Choose a new font, style, and size, then click **OK**. The font of the `Label` control will update to your selection.

Optional: Customize the Font Dialog

You can customize the `FontDialog` component by setting various properties, which can be done in the Properties window or in your code.

Example: Show the color selector

By default, the font dialog does not include a color selection dropdown. You can enable it with the `ShowColor` property.

1. Add a `ColorDialog` control from the Toolbox to your form.
2. Modify your `button1_Click` event to include the following code:

```
private void button1_Click(object sender, EventArgs e)
{
    // Enable the color selection in the font dialog
    fontDialog1.ShowColor = true;

    // Set the initial font and color to match the label
    fontDialog1.Font = label1.Font;
    fontDialog1.Color = label1.ForeColor;

    if (fontDialog1.ShowDialog() == DialogResult.OK)
    {
        label1.Font = fontDialog1.Font;
        label1.ForeColor = fontDialog1.Color;
    }
}
```

Other customizable properties

- `MaxSize` and `MinSize` : Restrict the maximum and minimum font sizes a user can select.
- `ShowEffects` : Enable or disable options for strikethrough and underline.

- **ShowApply** : Display an "Apply" button in the dialog.
- **FontMustExist** : Show a message box error if the user enters a font name that does not exist.

4. Open File Dialog:

To open a file dialog in a C# Windows Forms application, you can use the OpenFileDialog class from the System.Windows.Forms namespace.

Here's how to implement it:

- **Add a Button to your Form:** Drag a Button control from the Toolbox onto your Windows Form in the designer. This button will trigger the OpenFileDialog.
- **Create an Event Handler for the Button Click:** Double-click the button in the designer to create a Click event handler in your code-behind.
- **Implement the OpenFileDialog in the Event Handler:**

```
using System.Windows.Forms; // Ensure this namespace is included
using System.IO; // For file operations, if you intend to read the file

public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private void button1_Click(object sender, EventArgs e)
    {
        // Create an instance of the OpenFileDialog
        OpenFileDialog openFileDialog1 = new OpenFileDialog();

        // Set properties for the dialog (optional)
        openFileDialog1.InitialDirectory = "c:\\"; // Set initial directory
        openFileDialog1.Filter = "Text files (*.txt) | *.txt | All files (*.*) | *.*"; // Filter file types
        openFileDialog1.FilterIndex = 1; // Set the default filter (e.g., Text files)
        openFileDialog1.RestoreDirectory = true; // Restore the directory after closing
        openFileDialog1.Title = "Select a File"; // Set the dialog title

        // Show the dialog and check if the user clicked OK
        if (openFileDialog1.ShowDialog() == DialogResult.OK)
        {
```

```

        // Get the selected file name
        string selectedFilePath = openFileDialog1.FileName;

        // You can now use the selected file path, for example, to display it in a TextBox
        // textBox1.Text = selectedFilePath;

        // Or read the content of the file
        // string fileContent = File.ReadAllText(selectedFilePath);
        // MessageBox.Show(fileContent, "File Content");
    }
    else
    {
        // User clicked Cancel or closed the dialog
        MessageBox.Show("No file selected.", "Information");
    }
}
}

```

Explanation:

- `OpenFileDialog openFileDialog1 = new OpenFileDialog();`: This line creates a new instance of the `OpenFileDialog` class.
- `InitialDirectory, Filter, FilterIndex, RestoreDirectory, Title`: These properties allow you to customize the appearance and behavior of the file dialog.
- `openFileDialog1.ShowDialog()`: This method displays the file dialog to the user. It returns a `DialogResult` value indicating how the user closed the dialog (e.g., OK, Cancel).
- `if (openFileDialog1.ShowDialog() == DialogResult.OK)`: This condition checks if the user clicked the "Open" button in the dialog.
- `openFileDialog1.FileName`: If a file was selected, this property contains the full path of the chosen file.
- `System.IO.File.ReadAllText(selectedFilePath)`: This is an example of how you might read the content of the selected file using the `System.IO` namespace.

5. Save File Dialog:

To implement a "Save File" dialog in a C# Windows Forms application, you utilize the `SaveFileDialog` component. This component allows users to browse the file system and specify a file name and location to save data.

Here's a step-by-step guide and code example:

- **Add a Button to your Form:**

In the Visual Studio Designer, drag a Button control onto your form and give it a suitable name (e.g., btnSave) and text (e.g., "Save File").

- **Add a Click Event Handler:**

Double-click the button in the Designer to create a Click event handler in your code-behind file.

- **Implement the SaveFileDialog:**

Inside the button's Click event handler, you will instantiate and configure the SaveFileDialog and then handle the user's selection.

```
using System;
using System.IO; // Required for file operations
using System.Windows.Forms; // Required for SaveFileDialog

public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private void btnSave_Click(object sender, EventArgs e)
    {
        // Create a new SaveFileDialog instance
        SaveFileDialog saveFileDialog1 = new SaveFileDialog();

        // Set properties for the dialog
        saveFileDialog1.Filter = "Text files (*.txt)|*.txt|All files (*.*)|*.*"; // Filter for file types
        saveFileDialog1.FilterIndex = 1; // Default selected filter (Text files)
        saveFileDialog1.RestoreDirectory = true; // Restore the directory after closing
        saveFileDialog1.Title = "Save your file"; // Title of the dialog box
        saveFileDialog1.FileName = "MyDocument.txt"; // Default file name

        // Show the dialog and check if the user clicked OK
        if (saveFileDialog1.ShowDialog() == DialogResult.OK)
        {
            try
            {
                // Get the selected file path
                string filePath = saveFileDialog1.FileName;
            }
            catch { }
        }
    }
}
```

```

        // Example: Save some text content to the selected file
        string contentToSave = "This is some sample text to save.";
        File.WriteAllText(filePath, contentToSave);

        MessageBox.Show("File saved successfully!", "Success", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Error saving file: {ex.Message}", "Error",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
}
}

```

Explanation:

- `SaveFileDialog saveFileDialog1 = new SaveFileDialog();`: Creates an instance of the `SaveFileDialog`.
- `saveFileDialog1.Filter`: Defines the file types that the user can select, using a pipe-separated string. Each part specifies a description (e.g., "Text files") and a file extension pattern (e.g., *.txt).
- `saveFileDialog1.FilterIndex`: Sets the default selected filter in the dialog.
- `saveFileDialog1.RestoreDirectory`: Ensures that the dialog remembers the last directory accessed.
- `saveFileDialog1.Title`: Sets the title displayed in the dialog window.
- `saveFileDialog1.FileName`: Provides a default file name when the dialog opens.
- `saveFileDialog1.ShowDialog() == DialogResult.OK`: Displays the dialog box. The code inside the if block executes only if the user clicks the "Save" button (or equivalent) and not "Cancel".
- `string filePath = saveFileDialog1.FileName;`: Retrieves the full path and file name selected by the user.
- `File.WriteAllText(filePath, contentToSave);`: This is an example of how to save data to the chosen file. You would replace `contentToSave` and `File.WriteAllText` with your actual data and saving logic (e.g., saving an image, binary data, etc.).
- **Error Handling:** The try-catch block is crucial for handling potential exceptions during file operations (e.g., insufficient permissions, disk full).

6. MDI-Forms:

Building a Multiple Document Interface (MDI) application in C# involves creating a parent form that acts as a container for multiple child forms. This is a standard approach for applications that manage multiple documents or windows simultaneously, like Microsoft Excel or a code editor.

Step 1: Create the MDI parent form

1. **Start a new project:** Open Visual Studio and create a new project. Choose the "Windows Forms App" template for C#.
2. **Add a `MenuStrip`:** Drag a `MenuStrip` control from the Toolbox onto your main form (e.g., `Form1`). This is where you will add menu options for opening and managing child windows.
3. **Create the "Window" menu:** Add a top-level menu item named "Window" and set its `MdiWindowListItem` property to this new menu item. This tells the application to automatically populate this menu with a list of all open MDI child windows.
4. **Set `IsMdiContainer`:** Select your main form, go to the Properties window, and set the `IsMdiContainer` property to `True`. You will see the form's background change to a dark gray.
5. **Maximize the window (optional):** For better usability, set the `WindowState` property of the main form to `Maximized`.

Step 2: Create the MDI child form

1. **Add a new form:** Right-click your project in the Solution Explorer, select **Add > Windows Form**, and give the new form a descriptive name, like `ChildForm.cs`.
2. **Add child form content:** Use the visual designer to add any controls you want on the child form. For example, you can add a `RichTextBox` and set its `Dock` property to `Fill` so it resizes with the child window.

Step 3: Write code to open the child form

1. **Create a menu item to open the child form:** On your parent form's `MenuStrip`, add a new menu item under a "File" menu (e.g., a "New" item).

2. **Open the child on click:** Double-click the "New" menu item to create its `Click` event handler. Add the following code to open a new instance of your `ChildForm` as an MDI child:

```
private void newToolStripMenuItem_Click(object sender, EventArgs e)
{
    ChildForm newMDIChild = new ChildForm();
    // Set the parent of the child window.
    newMDIChild.MdiParent = this;
    // Display the new form.
    newMDIChild.Show();
}
```

Step 4: Implement window arrangement

1. **Add arrangement menu items:** Go back to your parent form's designer and add sub-menu items under your "Window" menu for arranging child windows (e.g., "Cascade," "Tile Horizontal," "Tile Vertical").
2. **Implement the arrangement logic:** Double-click each menu item to create its `Click` event handler and use the parent form's `LayoutMdi` method with the appropriate `MdiLayout` enumeration value. **Cascade:**

```
private void cascadeToolStripMenuItem_Click(object sender, EventArgs e)
{
    this.LayoutMdi(MdiLayout.Cascade);
}
```

Tile Horizontal:

```
private void tileHorizontalToolStripMenuItem_Click(object sender, EventArgs e)
{
    this.LayoutMdi(MdiLayout.TileHorizontal);
}
```

Tile Vertical:

```
csharp
private void tileVerticalToolStripMenuItem_Click(object sender, EventArgs e)
{
    this.LayoutMdi(MdiLayout.TileVertical);
}
```

Use code with caution.

Step 5: Run the application

Press **F5** to run your application. You can now use the menu to create and manage multiple child forms within the main parent window.

7. MenuBar:

The MenuStrip control is used to create a menu bar.

Creating a MenuStrip at Design-Time:

- **Open your Windows Forms project in Visual Studio.**
 - Drag and drop a MenuStrip control from the Toolbox onto your Form. It will typically dock to the top of the form automatically.
- **Add Menu Items:**
 - Click on the MenuStrip control on your form.
 - You will see a "Type Here" prompt. Type the name of your main menu item (e.g., "File", "Edit", "View").
 - Press Enter or click on the next "Type Here" area to add sub-items under the main menu item (e.g., "New", "Open", "Save" under "File").
 - Continue adding menu items and sub-items as needed.
- **Customize Menu Item Properties:**
 - Select a specific menu item.
 - In the Properties window, you can set properties like Text, Name, ShortcutKeys, Image, Enabled, Visible, etc.
- **Handle Click Events:**
 - Double-click on a menu item in the designer to automatically generate its Click event handler in the code-behind file.
 - Write the code to be executed when that menu item is clicked within the event handler.

Creating a MenuStrip at Run-Time (Dynamically):

You can also create and add a MenuStrip and its items programmatically.

using System.Windows.Forms;

```

public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
        CreateDynamicMenu();
    }

    private void CreateDynamicMenu()
    {
        // Create a new MenuStrip
        MenuStrip dynamicMenuStrip = new MenuStrip();
        this.Controls.Add(dynamicMenuStrip);

        // Create main menu items
        ToolStripMenuItem fileMenuItem = new ToolStripMenuItem("File");
        ToolStripMenuItem editMenuItem = new ToolStripMenuItem("Edit");

        // Add main menu items to the MenuStrip
        dynamicMenuStrip.Items.Add(fileMenuItem);
        dynamicMenuStrip.Items.Add(editMenuItem);

        // Create sub-items for "File"
        ToolStripMenuItem newMenuItem = new ToolStripMenuItem("New");
        ToolStripMenuItem openMenuItem = new ToolStripMenuItem("Open");
        ToolStripMenuItem saveMenuItem = new ToolStripMenuItem("Save");
        ToolStripMenuItem exitMenuItem = new ToolStripMenuItem("Exit");

        // Add sub-items to "File"
        fileMenuItem.DropDownItems.Add(newMenuItem);
        fileMenuItem.DropDownItems.Add(openMenuItem);
        fileMenuItem.DropDownItems.Add(saveMenuItem);
        fileMenuItem.DropDownItems.Add(new ToolStripSeparator()); // Add a separator
        fileMenuItem.DropDownItems.Add(exitMenuItem);

        // Attach event handlers
        newMenuItem.Click += NewMenuItem_Click;
        exitMenuItem.Click += ExitMenuItem_Click;
    }

    private void NewMenuItem_Click(object sender, System.EventArgs e)
    {
        MessageBox.Show("New file created!");
    }
}

```

```

private void ExitMenuItem_Click(object sender, System.EventArgs e)
{
    Application.Exit();
}
}

```

8. Context Menu:

The context menus are implemented using the ContextMenuStrip control. This control allows you to provide context-sensitive options to users when they right-click on a form or a specific control within it.

Steps to implement a ContextMenuStrip:

- Add ContextMenuStrip to your Form:
- Drag and drop a ContextMenuStrip control from the Toolbox onto your form in the Visual Studio designer. It will appear in the component tray at the bottom of the designer, not directly on the form.
- **Define Menu Items:**
 - Select the ContextMenuStrip in the component tray.
 - In the designer, a "Type Here" box will appear on the ContextMenuStrip. You can directly type the names of your menu items and sub-menu items here.
 - For each menu item, you can set its Text property, and optionally its Name property for easier reference in code.
 - Associate ContextMenuStrip with a Control:
 - Select the control (e.g., a Button, TextBox, or the Form itself) to which you want to attach the context menu.
 - In the Properties window for that control, locate the ContextMenuStrip property.
 - Select the ContextMenuStrip you added in step 1 from the dropdown list.
- **Handle Menu Item Clicks:**
 - Double-click on a menu item within the ContextMenuStrip in the designer. This will generate a Click event handler in your code-behind file.
 - Inside the event handler, write the C# code to perform the desired action when that specific menu item is clicked.

Example Code Snippet (for a simple "Exit" menu item):

```
// In your Form's constructor or Load event
public Form1()
{
    InitializeComponent();

    // Assuming you have a ContextMenuStrip named contextMenuStrip1
    // and a ToolStripMenuItem named exitToolStripMenuItem within it.
    // If setting programmatically:
    // ContextMenuStrip contextMenuStrip = new ContextMenuStrip();
    // ToolStripMenuItem exitItem = new ToolStripMenuItem("Exit");
    // exitItem.Click += ExitToolStripMenuItem_Click;
    // contextMenuStrip.Items.Add(exitItem);
    // this.ContextMenuStrip = contextMenuStrip; // Attach to the form
}

// Event handler for the "Exit" menu item
private void exitToolStripMenuItem_Click(object sender, EventArgs e)
{
    Application.Exit(); // Closes the application
}
```

9. MessageBox:

The MessageBox class from the System.Windows.Forms namespace is used to display simple dialog boxes to the user. These message boxes can convey information, ask for confirmation, or present warnings.

Basic Usage:

The simplest way to display a message box is using the static MessageBox.Show() method, providing only the message text:

```
MessageBox.Show("This is a simple message.");
```

Adding a Title:

You can include a title for the message box by providing a second string argument:

```
MessageBox.Show("This is a message with a title.", "My Application Title");
```

Customizing Buttons:

The `MessageBoxButtons` enumeration allows you to specify the buttons displayed in the message box (e.g., `OK`, `OKCancel`, `YesNo`, `YesNoCancel`, `AbortRetryIgnore`, `RetryCancel`).

```
DialogResult result = MessageBox.Show("Do you want to save changes?", "Confirmation",  
MessageBoxButtons.YesNoCancel);
```

```
if (result == DialogResult.Yes)  
{  
    // User clicked Yes  
}  
else if (result == DialogResult.No)  
{  
    // User clicked No  
}  
else if (result == DialogResult.Cancel)  
{  
    // User clicked Cancel  
}
```

Adding Icons:

The `MessageBoxIcon` enumeration enables you to display an icon relevant to the message type (e.g., `Information`, `Warning`, `Error`, `Question`).

```
MessageBox.Show("An error occurred.", "Error", MessageBoxButtons.OK,  
MessageBoxIcon.Error);
```

Setting a Default Button:

You can specify which button is the default (i.e., activated when the user presses Enter) using the `MessageBoxDefaultButton` enumeration.

```
MessageBox.Show("Are you sure?", "Confirm Action", MessageBoxButtons.YesNo,  
MessageBoxIcon.Question, MessageBoxDefaultButton.Button2); // Button2 refers to 'No'
```

Combining Options:

Multiple options can be combined in the `MessageBox.Show()` method:

```
DialogResult response = MessageBox.Show(  
    "Are you sure you want to delete this item?",  
    "Confirm Deletion",  
    MessageBoxButtons.YesNo,  
    MessageBoxIcon.Warning,  
    MessageBoxDefaultButton.Button2 // Default to 'No'  
);
```

```
if (response == DialogResult.Yes)
{
    // Perform deletion
}
```

Important Considerations:

- `MessageBox.Show()` is a static method, so you do not need to create an instance of the `MessageBox` class.
- The `DialogResult` enumeration is crucial for determining which button the user clicked in message boxes with multiple buttons.
- For more complex or highly customized dialogs, consider creating a custom Form instead of relying solely on `MessageBox`.

10. InputBox:

In C#, you can create an input box in a Windows Forms application in two main ways: by creating a custom dialog form or by referencing the `Microsoft.VisualBasic` library. A custom dialog is the recommended approach for modern C# applications, as it provides more flexibility and doesn't require adding a Visual Basic dependency.

Method 1: Create a custom dialog form

This is the most common and robust method, allowing you to create a reusable class that functions like a standard `InputBox`.

1. Add a new Windows Form

In Visual Studio, right-click your project in the Solution Explorer, and select **Add > Form (Windows Forms)**. Name the form `InputDialog.cs`.

2. Design the dialog form

Add the following controls to your `InputDialog` form:

- A `Label` to display a message to the user.
- A `TextBox` for user input.
- An "OK" `Button`.

- A "Cancel" Button .

3. Configure the form and controls

- Set the `FormBorderStyle` property of the form to `FixedDialog` .
- Set the `AcceptButton` property of the form to your "OK" button.
- Set the `CancelButton` property of the form to your "Cancel" button.
- Set the `DialogResult` property of the "OK" button to `OK` and the "Cancel" button to `Cancel` .

4. Add code to the dialog form

Open the code-behind file for `InputDialog.cs` and add a static method to show the dialog and return the result.

```
using System;
using System.Windows.Forms;

public partial class InputBoxDialog : Form
{
    public string UserInput { get; private set; }

    public InputBoxDialog(string prompt, string title, string defaultValue = "")
    {
        InitializeComponent();
        this.Text = title;
        this.lblPrompt.Text = prompt;
        this.txtInput.Text = defaultValue;
    }

    public static string Show(string prompt, string title, string defaultValue = "")
    {
        using (var form = new InputBoxDialog(prompt, title, defaultValue))
        {
            DialogResult result = form.ShowDialog();
            if (result == DialogResult.OK)
            {

```

```

        return form.UserInput;
    }
    return null; // Return null if the user cancels
}
}

private void btnOk_Click(object sender, EventArgs e)
{
    this.UserInput = txtInput.Text;
    this.DialogResult = DialogResult.OK;
    this.Close();
}
}

```

5. Use the custom dialog

From your main form, call the static `Show` method you created to display the input box and retrieve the result.

```

csharp
private void myButton_Click(object sender, EventArgs e)
{
    string userInput = InputBoxDialog.Show("Please enter your name:", "User Input", "John Doe");

    if (userInput != null)
    {
        MessageBox.Show($"Hello, {userInput}!");
    }
    else
    {
        MessageBox.Show("Operation cancelled.");
    }
}
}

```

Method 2: Use the Microsoft.VisualBasic library

This method provides a simpler, single-line call, but it requires adding a reference to a Visual Basic library.

1. Add a reference to `Microsoft.VisualBasic`

- In the Solution Explorer, right-click on your project's **Dependencies** or **References** and select **Add Reference**.
- Navigate to **Assemblies > Framework** and select `Microsoft.VisualBasic`.
- Click **OK**.

2. Call the `InputBox` method

You can now call the `InputBox` method from the `Microsoft.VisualBasic.Interaction` class.

```
using Microsoft.VisualBasic;

private void myButton_Click(object sender, EventArgs e)
{
    string userInput = Interaction.InputBox("Please enter your name:", "User Input", "John Doe");

    if (!string.IsNullOrEmpty(userInput))
    {
        MessageBox.Show($"Hello, {userInput}!");
    }
    else
    {
        MessageBox.Show("Operation cancelled.");
    }
}
```